

From Empirical Evaluation to Context-Aware Enhancement: Repairing Regression Errors with LLMs

ANH HO, The University of Melbourne, Australia

THANH LE-CONG, Singapore University of Technology and Design, Singapore

BACH LE, The University of Melbourne, Australia

CHRISTINE RIZKALLAH, The University of Melbourne, Australia

Software systems constantly evolve to adapt to ever-changing customer demands and evolving markets. During the software evolution process, code changes across versions can unexpectedly break previously working functionalities, leading to so-called regression bugs. Manually repairing regression bugs is both challenging and time-consuming, as the process often requires developers to traverse the history of the software in order to understand how the bugs were introduced and, consequently, how to repair them. Despite the challenges of manual efforts, only a limited number of studies have investigated automated program repair (APR) for regression bugs, most of which date back several years and focus on traditional APR techniques. Since then, various APR approaches, especially those leveraging the power of large language models (LLMs), have been rapidly developed to fix general software bugs. Unfortunately, the effectiveness of these advanced techniques in the context of regression bugs remains largely unexplored. This gap motivates the need for an empirical study evaluating the effectiveness of modern APR techniques in fixing real-world regression bugs.

In this work, we conduct an empirical study of APR techniques on regression bugs. To facilitate our study, we introduce REGRESSIONBUG4APR, a high-quality benchmark of Java and Python regression bugs integrated into a framework designed to facilitate APR research. The current benchmark includes 200 regression bugs collected from widely used real-world GitHub repositories. We begin by conducting an in-depth analysis of the benchmark, demonstrating its diversity and quality. Building on this foundation, we empirically evaluate the capabilities of APR to regression bugs by assessing both traditional APR tools and advanced LLM-based APR approaches. Our experimental results show that classical APR tools fail to repair any bugs, while LLM-based APR approaches exhibit promising potential. Motivated by these results, we investigate impact of incorporating bug-inducing change information into LLM-based APR approaches for fixing regression bugs. We further conduct an ablation study to disaggregate the contribution of each contextual element within the bug-inducing change information. Our results highlight that this context-aware enhancement significantly improves the performance of LLM-based APR, yielding $1.6\times$ more successful repairs compared to using LLM-based APR without such context. Moreover, our findings are consistent across both Java and Python benchmarks, providing preliminary evidence for the generalizability of our findings.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **General and reference** → **Empirical studies**.

Additional Key Words and Phrases: Regression Bugs, Regression Repair, Automated Program Repair, Benchmark, Large Language Models

Authors' Contact Information: Anh Ho, The University of Melbourne, Australia, anh.ho1@student.unimelb.edu.au; Thanh Le-Cong, Singapore University of Technology and Design, Singapore, congthanh_le@sutd.edu.sg; Bach Le, The University of Melbourne, Australia, bach.le@unimelb.edu.au; Christine Rizkallah, The University of Melbourne, Australia, christine.rizkallah@unimelb.edu.au.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

ACM Reference Format:

Anh Ho, Thanh Le-Cong, Bach Le, and Christine Rizkallah. 2025. From Empirical Evaluation to Context-Aware Enhancement: Repairing Regression Errors with LLMs. 1, 1 (June 2025), 59 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Software systems are continuously evolving to meet user demands and adapt to rapidly changing market conditions, resulting in increasing system complexity and frequent source code modifications. Each modification usually requires interactions with an ever-growing number of components within the code base such as calling to existing functionality or integrating with other parts of the system. However, these code changes can unintentionally introduce regression bugs, disrupting previously working features of software systems [66].

Regression bugs often stem from the unintended consequences of modifications to existing code. This unique characteristic makes regression bugs particularly challenging, as developers must identify the specific changes responsible, i.e., bug-inducing changes, and trace the effects of bug-inducing changes on previously functioning features [83]. In large-scale software projects, the challenges associated with regression bugs are intensive, as they often remain undetected for several years before being identified and resolved [13, 81]. Indeed, empirical studies highlight the significant prevalence of regression bugs in real-world systems. Xiao et al. [75] conducted a large-scale empirical study of 5,741 bug reports from the Linux kernel and found that approximately half of all bugs are regressions. Similarly, Khattar et al. [29] reported that the proportion of regression bugs in the Google Chromium project reaches 51.09%. This underscores that regression bugs are not a niche concern but a dominant and persistent category of defects in large-scale real-world systems. Furthermore, Xiao et al. [74] studied 1,579 regression bugs across 57 Linux versions and found that bugs occurring in regression bug chains cost $2.4\times$ more fixing time, involve $1.3\times$ more developers, and generate $2.8\times$ more discussion comments compared to isolated regression bugs, with 68% of such chains propagating across software versions and compounding in cost over time. Moreover, low-quality fixes might introduce new regressions, creating a recurring maintenance cycle that further amplifies the overall cost and effort [12, 74]. Consequently, regression bugs continue to be a persistent challenge in the software industry, affecting overall software quality and maintenance efforts [19, 53].

Automated program repair (APR) techniques were developed to reduce the manual effort required to fix bugs and improve software quality by automating the repair process. Hence, applying APR to regression bugs would be particularly valuable, given their complexity and the effort required to address them. To date, there has been extensive research on APR for general software bugs [21, 37, 48], including a growing interest in large language model (LLM)-based APR techniques in recent years [14, 23, 24, 38, 43, 57, 60, 76, 78, 80]. However, the most recent work specifically targeting regression bug repair dates back several years. Notably, Relifix, developed for C/C++ programs, focuses solely on traditional pattern-based and search-based APR techniques [66]. This naturally raises an important question:

How well do state-of-the-art APR techniques perform when applied specifically to regression bugs?

This motivates the need for an empirical study to revisit existing APR techniques to better understand their effectiveness on regression bugs, where they fall short and can be further improved.

One major challenge in conducting the empirical study lies in the lack of a high-quality and up-to-date benchmark that reflects both current software development practices and the ongoing progress in this research field. Popular datasets commonly used for evaluating APR techniques, such as Defects4J [27], Bugs.jar [58] and Bears [44], primarily

target general software bugs and are associated with bug-triggering test cases that have been added/updated after the bug is reported [40]. These datasets are not constructed with regression-specific characteristics in mind. The bugs are not revealed through regression testing, making it unclear whether they truly represent regressions. Hence, these datasets lack the temporal and behavioral properties that define regression bugs. Existing datasets for regression bugs, such as CoREBench [13] and CIBugs [28], are outdated (bugs are before 2017) and difficult to reproduce, making them less suitable for studying regression repair.

To address these problems, we propose REGRESSIONBUG4APR, a high-quality Java and Python regression error benchmark integrated into a framework designed to facilitate APR research. For Java, the benchmark is built upon an automated tool that mines regression bugs from software evolution history, namely REGMINER [64]. By leveraging REGMINER, we automatically collect potential regression errors from Java code repositories and then validate the regressions. For Python, we developed a dedicated validation tool following the same high-level design as RegMiner, adapted for the Python ecosystem. Our benchmark collection framework ensures the reproducibility and quality of each mined bug instance, while also supporting future benchmark extensions. Through this automated process, we have established REGRESSIONBUG4APR, which includes 200 confirmed regression bugs collected from widely used real-world Java and Python GitHub repositories.

We conduct an in-depth analysis of our benchmark REGRESSIONBUG4APR to demonstrate desired properties of the benchmark, including: *reality*, where regression bugs are extracted from diverse real-world open-source software; *currency*, with the bugs spanning from 2016 onward; *diversity*, encompassing a broad range of projects and diverse repair operators; *extensibility*, as our approach enables the dataset to expand over time with minimal human efforts by building upon REGMINER’s approach [64]; and *durable replicability*, with the framework offering easy access, extensibility, and snapshots of all projects and their dependencies, similar to that of Defects4J [27]. Prior study suggested that these properties are often required to enable fair evaluations [68].

Using the diverse and high-quality benchmark, we conduct an empirical study to revisit APR techniques on regression errors. The evaluation is broadly categorized into two classes: *traditional* and *advanced LLM-based* techniques. For *traditional* techniques, we examine template-based and search-based tools, including GenProg [36], Kali [55], Cardumen [46], Arja [82], jMutRepair [45], and TBar [39]. For *advanced* techniques, we focus on LLM-based approaches, further divided into two groups: *fine-tuning-based* and *prompt-based* techniques. In the *fine-tuning* category, we evaluate full fine-tuning methods, including CodeGen models with 2B and 6B parameters and Incoder models with 1B and 6B parameters [71], as well as parameter-efficient fine-tuning methods, such as RepairLLaMA [60]. For *prompt-based* approaches, we explore two repair strategies, *zero-shot prompting* and *conversational interaction*, using advanced general-purpose LLMs: ChatGPT-3.5-Turbo and ChatGPT-4o. Note that traditional and fine-tuning-based techniques are evaluated on Java regression bugs only, as they are inherently designed for Java and do not support Python. To ensure a fair comparison, prompt-based techniques are also evaluated on the Java regression bugs when compared against these approaches. Our experiments show that traditional pattern-based and search-based APR tools fail to repair any regression bugs in our dataset. In contrast, LLM-based approaches demonstrate promising performance: RepairLLaMA, the top-performing fine-tuning-based technique, successfully repairs 15 bugs, while ChatGPT-4o with conversational prompting, the most effective prompt-based method, successfully repairs 18 bugs on the Java benchmark. It is, however, noted that these LLM-based techniques are not inherently designed to tackle the nature of regression bugs, e.g., taking into account the bug-inducing changes to inform bug repair and thus leaving room for further improvements.

Finally, we further explore the impact of incorporating additional regression-specific context, e.g., bug-inducing change information, on improving the repair effectiveness of LLM-based APR. We do so by leveraging prompt-based

approaches, explicitly integrating bug-inducing changes into the prompt design. This choice is driven by the inherent flexibility of prompt-based methods in terms of model usage and prompt construction, as well as their data efficiency compared to fine-tuning-based techniques. Our experiments show that incorporating this additional context significantly enhances the performance and precision of LLM-based techniques using prompting. Specifically, the best-performing configuration, conversational ChatGPT-4o with full bug-inducing change (BIC) information, successfully repairs 39 out of 200 regression bugs in the REGRESSIONBUG4APR benchmark, representing a $1.6\times$ improvement over the best-performing configuration without BIC context. Interestingly, our qualitative analysis further reveals that bug-inducing change information provides crucial context that helps models better identify the root cause of the regression and determine whether to fully or partially revert to previous correct statements. In addition, it helps narrow the fix scope, thereby improving fault localization and reducing unnecessary code modifications.

To further understand the contribution of each contextual element, we conduct an ablation study. Our results reveal that each component contributes positively and incrementally to repair performance, with code-level changes providing a larger gain than commit messages alone. Notably, commit messages and code changes capture complementary, non-redundant signals that together enhance repair effectiveness.

In summary, the key contributions of this work are as follows:

- 1) We introduce REGRESSIONBUG4APR, a high-quality Java and Python regression error benchmark integrated into a framework that supports reproducible studies in automated program repair and enables future extensions. We also provide a comprehensive analysis of the benchmark’s characteristics.
- 2) We perform an empirical study evaluating the effectiveness of several APR techniques, including recent state-of-the-art LLM-based approaches, on regression bugs.
- 3) We investigate the impact of incorporating bug-inducing change information on the effectiveness of prompt-based repair techniques. Our study includes a qualitative analysis that reveals how this regression-specific context supports models in fixing regression errors.
- 4) We conduct an ablation study to investigate the individual contribution of each contextual element within the bug-inducing change information.

Replication Packages: To support further research, we provide all artifacts associated with our dataset in a reproducible and easily accessible form. Moreover, the REGRESSIONBUG4APR benchmark is designed for extensibility, allowing the database to grow over time with minimal human effort. All artifacts related to this work are publicly available at the following links [2–7]. In addition, the implementation of the prompt-based APR techniques for regression bugs used in our experiments is also publicly available [1].

The remainder of this paper is organized as follows. Section 2 provides background information and discusses related work on regression bugs and automated program repair. Section 3 describes the study setup, including the research and experimental designs. Section 4, 5, 6 and 7 report the experimental results, followed by Section 8, which explores the implications of our findings and the threats to validity. Finally, Section 9 summarizes our conclusions and outlines directions for future work.

2 Background and Related Work

In this section, we first introduce the concept of regression bugs, including their characteristics, categorization, and concrete examples for each type, along with existing benchmarks and tools for mining regression bugs. We then discuss related work on program repair benchmarks and automated program repair (APR) techniques. Finally, we provide an overview of regression bug repair, along with an illustrative example and formal problem formulation.

2.1 Regression Bug

Regression bug is a common class of bugs that occur when a feature that previously worked correctly stops functioning after certain changes, such as bug fixes, new feature additions, code refactoring, or other software modifications. A regression test is designed to ensure that existing functionality or intended behavior remains intact during software evolution. When such a test fails, indicating that a recent change has unintentionally reverted the software to a “less developed state”, the associated bug is referred to as a *regression bug* [51].

2.1.1 Categorization of Regression Bug. To better understand the different causes of regression bugs, Tan et al. [66] categorizes regression bugs into three types based on how the regression was introduced:

(i) *Local*. A code modification directly breaks existing functionality within the modified program element. Intuitively, a local regression can often be fixed by reverting to the previous implementation. However, reverting may unintentionally undo newly intended updates associated with the modification. RegressionBug-40 from *jsoup*¹ project, a Java HTML/XML parsing library, illustrates this type. The bug-inducing commit modified one line in `popStackToClose()`, which matches a closing XML tag to its open element on the parse stack:

```
@@ -114,7 +114,7 @@
* @param endTag tag to close
*/
private void popStackToClose(Token.EndTag endTag) {
-   String elName = endTag.name();
+   String elName = endTag.normalName();
   Element firstFound = null;

   for (int pos = stack.size() - 1; pos >= 0; pos--) {
```

Fig. 1. Bug-inducing changes in *RegressionBug-40* from our REGRESSIONBUG4APR benchmark.

This single-line change directly broke the method’s own logic. Open elements are pushed onto the stack using `settings.normalizeTag()`, which preserves the tag name case when the XML parser is configured to do so (the default, since XML is case-sensitive). After the bug-inducing commit, a closing tag `</FOO>` is resolved via `normalName()`, which unconditionally returns “foo”. The subsequent name comparison against the stack element “FOO” fails, `firstFound` remains null, and the method returns without closing the element, corrupting the parse tree. The failure is entirely confined to `popStackToClose()` itself; the modified method is the one that breaks, with no other program element involved and no pre-existing latent fault triggered. The bug-fixing commit restored correctness by replacing `endTag.normalName()` with `settings.normalizeTag(endTag.tagName)`, ensuring that closing tags are normalized using the same parser settings as opening tags. This is the characteristic of a local regression: the modification directly introduces an incorrect behavior within the modified program element.

(ii) *Remote*. A code modification introduces a bug in an unchanged program element, leading to failures elsewhere in the codebase. RegressionBug-56 from *overthere*² project, a Java library for remote file manipulation and command execution, illustrates this type. The bug-inducing commit modified the sanitize loop in `escapeSpecialCharacters()` to escape the % character by doubling it, a standard technique to prevent Windows shell variable expansion:

This change is confined to the `sanitize` method. However, it introduced a regression in the entirely separate and unchanged method `containsAnySpecialChars()`, which contains the following check: `if (str.indexOf(PERCENT_CHAR) >= 0) {return true;}`.

¹Available at: <https://github.com/jhy/jsoup>

²Available at: <https://github.com/xebialabs/overthere>

```
+         if (c == '%') {
+             builder.append('%');
+         }
```

Fig. 2. Bug-inducing changes in *RegressionBug-56* from our REGRESSIONBUG4APR benchmark.

Before the commit, this check was correct, % was not handled by the sanitizer, so flagging it in `containsAnySpecialChars()` correctly informed callers that strings containing % required special treatment such as quoting. After the commit, % is properly escaped by the sanitizer, making the check in `containsAnySpecialChars()` incorrect: it continues to flag % as requiring special treatment, causing callers to apply additional escaping to strings that the sanitizer already handles, resulting in double-escaping. The bug-fixing commit restored correctness by removing the now-redundant % check from `containsAnySpecialChars()`:

```
-         if (str.indexOf(PERCENT_CHAR) >= 0) {
-             return true
-         }
```

Fig. 3. Bug-fixing changes in *RegressionBug-56* from our REGRESSIONBUG4APR benchmark.

This is a remote regression because the failure occurs in `containsAnySpecialChars()`, a program element that was not modified by the bug-inducing commit. Unlike an unmask regression, there is no pre-existing latent bug; the method was correct before the commit but was not updated to account for the new handling of % introduced by the sanitize modification, causing incorrect behavior elsewhere in the codebase. This illustrates a scenario where a developer modifies one part of the system without anticipating the assumptions that other unchanged parts depend upon.

(iii) *Unmask*. A code modification exposes an existing bug that had previously not affected the behavior of some tests. For an unmask regression, where the earlier code effectively concealed an underlying bug, fixing the issue may involve restoring conditions that mask the problematic changes. In such cases, repair efforts focus on finding a scenario where the changes no longer impact test behavior, without simply reverting to the previous code, as doing so would re-hide the bug and fail to address the underlying problem. *RegressionBug-132* from *univocity-parsers*³ project, a Java CSV/TSV parsing library, illustrates this type. The bug-inducing commit refactored `handleException()` in `AbstractParser` to enrich error diagnostics, changing it from a void method that throws to a method that returns the constructed exception.

```
- throw new TextParsingException(context, message, ex);
+ ...
+ return new TextParsingException(context, message, ex);
```

Fig. 4. Bug-inducing changes in *RegressionBug-132* from our REGRESSIONBUG4APR benchmark: refactoring of `handleException()`.

The call site was updated accordingly:

```
- handleException(ex);
+ ex = handleException(ex);
```

Fig. 5. Bug-inducing changes in *RegressionBug-132* from our REGRESSIONBUG4APR benchmark: update to the `handleException()` call site.

Both the method and its call site were updated consistently. The change is correct and complete for its stated purpose. The latent fault resided in the pre-existing, untouched method `stopParsing(Throwable error)`, where the re-throw logic was placed inside a catch block rather than unconditionally after it:

³Available at: <https://github.com/univocity/univocity-parsers>

```

try {
    stopParsing ();           // release resources
} catch (Throwable ex) {
    if (error instanceof RuntimeException)
        throw (RuntimeException ) error ;
    ... // re - throw only if cleanup itself throws
}

```

Fig. 6. Pre-existing latent fault in `stopParsing(Throwable error)` in *RegressionBug-132* from our REGRESSIONBUG4APR benchmark.

Before the bug-inducing commit, `handleException(ex)` threw a `TextParsingException`. Since an exception was already propagating when the `finally` block ran, `stopParsing(ex)` executed as part of cleanup, after which the `TextParsingException` from `handleException()` continued propagating to the caller regardless of what occurred inside `stopParsing()`. The latent fault in `stopParsing()` was therefore harmless: parsing errors always reached the caller via `handleException()`'s own throw.

After the bug-inducing commit, `handleException()` returns instead of throwing, so no exception is in flight when the `finally` block runs. The `TextParsingException` is now passed as an argument to `stopParsing(ex)`. When `stopParsing()` completes without itself throwing, the pre-existing fault causes the `TextParsingException` to be silently discarded, leaving the caller with no indication that parsing failed. The bug-fixing commit corrected `stopParsing(Throwable error)` by moving the re-throw logic outside the catch block so it always executes after cleanup:

```

    } catch (Throwable ex) {
-       if (error instanceof RuntimeException) throw (RuntimeException ) error ;
-       ...
    }
+   if (error instanceof RuntimeException) throw (RuntimeException ) error ;
+   ...

```

Fig. 7. Bug-fixing change in *RegressionBug-132* from our REGRESSIONBUG4APR benchmark.

This is an unmask regression because the failure originates in `stopParsing(Throwable error)`, a method not modified by the bug-inducing commit whose fault pre-dated it entirely. Unlike a local regression, the bug-inducing change is not itself incorrect: refactoring `handleException()` to return rather than throw is a valid and complete refactoring. Unlike a remote regression, the fault in `stopParsing(Throwable error)` was not introduced in response to the new handling; it was present from the start, shielded from consequences by the exception propagation path that `handleException()`'s throw provided. The bug-inducing commit is innocent: it inadvertently removed that shield, exposing the dormant fault.

2.1.2 Regression Bug Benchmark. To facilitate regression error research, two notable datasets have been developed for studying regression bugs: CoREBench [13], which was used by Relifix, and the more recent CIBugs [28]. CoREBench, introduced in 2014, comprises a collection of 70 regression errors extracted from four small to medium open-source C/C++ projects (less than 35KLOC): Make, Grep, Findutils, and Coreutils, which are publicly accessible through GNU homepage⁴. The dataset has not been updated since its release. Moreover, due to quality concerns, such as dissatisfaction with how regression errors were defined and challenges with reproducibility, Relifix ultimately utilized only a subset of the dataset in its evaluation [66]. The more recent benchmark, CIBugs, utilizes data from continuous integration

⁴Available at: <http://savannah.gnu.org> and <http://debbugs.gnu.org>

(CI) systems. CIBug’s strategy leverages the TravisTorrent database [11], which contains metadata from 2,640,825 Travis builds synthesized from Travis CI’s API and build logs, last updated on February 8, 2017. The regression errors captured in CIBugs are identified through continuous integration testing, where existing tests (i.e., regression tests) that previously passed begin to fail after code changes. However, CIBugs lacks information on bug-inducing commits necessary for forming regression bugs, and identifying these commits remains a challenging task [9]. Furthermore, CIBugs requires strenuous human engineering efforts to manually curate the bugs, rendering it hard to scale.

2.1.3 RegMiner - A Regression Bug Mining Tool. While existing datasets rely heavily on manual curation, RegMiner offers an automated approach for identifying regression bugs from software evolution history [64]. Specifically, RegMiner constructs a regression bug by searching in code repositories for a bug-fixing commit (*bfc*), a bug-inducing commit (*bic*), and a test case (*t*) such that *t* passes on *bfc* and a version before *bic*, but fails on *bic* and a version before *bfc*.

To begin, RegMiner collects bug-fixing commits that introduce new test cases from version histories. A commit is confirmed as a bug-fixing commit if the added test case passes on the current version and fails on the version before it, which serves as a prerequisite to search the regression bug. RegMiner then applies a measurement heuristic to prioritize bug-fixing commits that are more likely to be regressions. Next, for each candidate *bfc*, RegMiner searches for a corresponding bug-inducing commit *bic*, where the test case fails on *bic* but passes on the version before *bic*. If such a commit is found, a regression instance is recorded as a tuple (*bfc*, *bic*, *t*).

This process encounters several technical challenges: (i) *Futile search on non-regression bug-fixing commits*: Searching through commit history is time-consuming, so RegMiner introduces a heuristic to quantify the likelihood of a commit being a regression-fixing commit; (ii) *Test dependency migration*: Verifying whether a test fails or passes on prior versions is complicated as code and dependency changes over time, especially when regressions were introduced years earlier and the project has since undergone significant refactoring or library updates; and (iii) *High overhead for validation*: Confirming a regression requires checking out, compiling, and running tests across potentially thousands of commits, which incurs significant computational cost.

To overcome these challenges, RegMiner introduces three algorithms: (i) a heuristic for estimating potential bug-fixing commits, (ii) a test migration algorithm, and (iii) a validation effort minimization algorithm. These strategies enable scalable mining. The *validation effort minimization* algorithm is a heuristic designed to reduce validation costs and to handle cases that suffer from incompatibility due to test migration. However, this approach can lead to incorrect identification of regression bugs. To address this problem, we develop a validation tool to ensure the reproducibility, correctness, and the overall quality of each mined regression instance.

2.2 Program Repair

The following is an overview of existing program repair benchmarks and of automated program repair methodologies.

2.2.1 Program Repair Benchmarks. High-quality datasets are critical for evaluating the effectiveness of APR techniques and facilitating rigorous comparisons across different approaches. Over time, researchers have introduced various benchmarks to meet these needs.

Defects4J [27], one of the most widely used APR benchmarks, contains 835 real-world bugs from Java projects. Bugs.jar [58], which includes 1,158 reproducible bugs collected from Apache projects, follows a similar construction process but expands the benchmark’s size and covers different categories of Java applications. Bears [44] and BugSwarm [68] use automated processes to construct their datasets but differ in how bugs are reproduced. Bears includes 251 reproducible bugs, while BugSwarm collects unprocessed bugs from failed CI builds, yielding 3,091 pairs of

failing and passing builds. However, a critical analysis [17] shows that only 50 Java and 62 Python bugs in BugSwarm are actually suitable for evaluating automated fault localization or program repair techniques. A recent study [40] indicated that approximately 90% of the bugs in Defects4J, Bugs.jar, and Bears are associated with test cases that were added or modified after the bugs were reported. These datasets are not constructed with regression-specific characteristics in mind. The bugs are not revealed through regression testing, making it unclear whether they truly represent regressions. Therefore, these datasets lack the temporal and behavioral properties that define regression bugs.

In addition to Java benchmarks, several datasets have been developed for other programming languages. For C/C++, ManyBugs [35] includes 185 bugs from nine large, popular open-source programs. IntroClass [35] contains 998 student-written programs with bugs, collected from small programming assignments in an introductory course. Codeflaws [67] includes 3,902 bugs extracted from Codeforces programming contests. Other notable benchmarks include BugsInPy [70] for Python and BugsJS [22] for JavaScript.

In contrast to these general software bug benchmarks, our proposed benchmark, REGRESSIONBUG4APR, is specifically focused on regression bugs. It includes regression-revealing test cases, along with the corresponding bug-inducing and bug-fixing commits. Unlike non-regression benchmarks, where each bug is typically identified by one or more failing test cases, REGRESSIONBUG4APR includes both a historical passing version and a subsequent failing version. This specificity distinguishes our benchmark and provides a valuable resource for understanding the characteristics of real-world regression bugs.

2.2.2 Automated Program Repair. Traditional APR techniques can be broadly categorized into three main groups: *search-based*, *template-based*, and *constraint-based* approaches. *Search-based* and *Template-based approaches* search for program variants that retain the required functionality while addressing the identified bug. Notable examples leveraging genetic programming include GenProg [36], HDRepair [31], Arja [82], and jMutRepair [45]. Another common strategy leverages predefined fix templates, either manually defined or mined from code repositories. Notable examples are TBar [39] and Cardumen [46]. Relifix [66], applied to C/C++ regression errors, reduces the search space by applying code transformations based on syntactic change patterns observed across multiple software versions. *Constraint-based approaches* represent the space of patch candidates using symbolic constraints. By encoding the desired program behavior as a set of logical constraints, these approaches synthesize code that satisfies both the specification and the test suite. Notable techniques are SemFix [49], Nopol [77], Angelix [47], and S3 [30]. In our study, we do not evaluate semantic-based APR approaches as they often require the inclusion of manually-written models for system calls whose source codes are not available.

Shifting to *deep learning-based* techniques, these methods are typically trained from scratch on large bug-fix corpora to automatically learn repair patterns from human-written patches. Once trained, the models can be applied to generate patches for buggy programs. Recent state-of-the-art techniques are CURE [26], RewardRepair [79], Recoder [84], and KNOD [25], NeverMore [8], among others. More recently, the rapid progress of large language models (LLMs) has given rise to a new line of *LLM-based APR techniques*, which have demonstrated substantial improvements over prior deep learning-based methods [24]. Due to their superior capabilities, our study places significant emphasis on LLM-based APR. These LLM-based methods typically utilize large pretrained model for code generation and can be grouped into two main categories: *fine-tuning-based* and *prompt-based* techniques. *Fine-tuning-based approaches* focus on refining the weights of code-specific LLMs, such as CodeLLama [56], Incoder [20] and CodeGen [50], to improve performance on bug-fixing tasks. Notably, Jiang et al. [24] demonstrated that full-parameter fine-tuning significantly enhances APR performance, achieving remarkable improvements over the base models. Silver et al. [60] investigated the effectiveness

of parameter-efficient fine-tuning using QLoRA [16]. Their work evaluated various input-output formatting strategies and introduced RepairLLama, a fine-tuned version of CodeLLaMA tailored specifically for program repair. *Prompt-based approaches* aim to directly leverage LLMs without finetuning, mainly via prompting. They design and evaluate different prompting strategies to provide repair information to the model. Xia et al. [72] proposed ChatRepair, a conversation-driven APR approach that interacts with ChatGPT using contextual feedback such as test names, relevant test code, and error messages to iteratively guide the repair process. Yin et al. [80] applied chain-of-thought prompting and few-shot learning techniques to further enhance the performance of LLMs on APR tasks.

2.3 Regression Bug Repair

2.3.1 Overview of Regression Bug Repair. Regression bug repair is a specialized form of test-based repair that targets regression bugs specifically. Similar to general-purpose test-based repair, it takes a buggy program and a test suite as input and aims to generate a patch that satisfies all tests. What distinguishes regression repair from general-purpose repair is that it is permitted to leverage additional regression-specific context, most notably, bug-inducing change information that general-purpose approaches do not consider.

This additional context is valuable because regression bugs do not introduce entirely new faults; rather, they violate previously correct functionality by modifying code that was once working correctly. Unlike general bugs, each regression bug is linked to a bug-inducing commit that records the code changes introduced at the point where the regression was first observed. Identifying this commit provides context about why the failure occurred, enabling repair techniques to distinguish unintended behavioral deviations from intended modifications and to reason about how the regression was introduced. Prior work has shown that many bug-fixing changes correspond to reverting or modifying the changes introduced in the bug-inducing commit [66, 69], suggesting that such information provides valuable guidance for automated repair. In practice, developers routinely use tools such as `git bisect`⁵ to identify bug-inducing commits. For example, in *PyRegression-2* in our benchmark, Issue #62520⁶ of the pandas project, a regression is reported where `Series.pow` works correctly in pandas 2.x but fails in pandas 3.x. A core contributor applies `git bisect` and identifies commit `e4ca40511c` (“*API: mode.nan_is_na to consistently distinguish NaN-vs-NA*”) as the bug-inducing commit. Based on this information, contributors narrow the investigation to the specific changes in that commit and involve the relevant code owner. The code owner identifies the faulty line in `ArrowEA._arith_method` and implements a fix in PR #62572⁷. This illustrates a clear workflow: (1) identifying the bug-inducing commit, (2) narrowing down the faulty changes, and (3) generating a fix guided by that context.

Compared to general-purpose bug repair, regression repair introduces both opportunities and challenges. On the opportunity side, the availability of a bug-inducing commit provides richer contextual information to guide the repair process. On the challenge side, regression bug repair introduces difficulties that do not arise in general settings. First, repair techniques must reason across multiple program versions to understand how recent modifications introduced the regression. Second, not all changes in a regression-inducing commit are incorrect; some are intended improvements. Repair approaches must carefully identify which changes are responsible for the regression while preserving valid modifications. For example, blindly reverting all changes in a bug-inducing commit may undo intended improvements alongside the faulty ones, leading to semantically incorrect patches.

⁵Available at: <https://git-scm.com/docs/git-bisect>

⁶Available at: <https://github.com/pandas-dev/pandas/issues/62520>

⁷Available at: <https://github.com/pandas-dev/pandas/pull/62572>

2.3.2 An Illustrative Example. We present an example of a regression bug, illustrating how code changes unexpectedly create errors while introducing new features or bug fixes. This example, corresponding to *RegressionBug-42* in our benchmark, comes from the *jsoup* project, a Java library designed to simplify working with real-world HTML and XML.

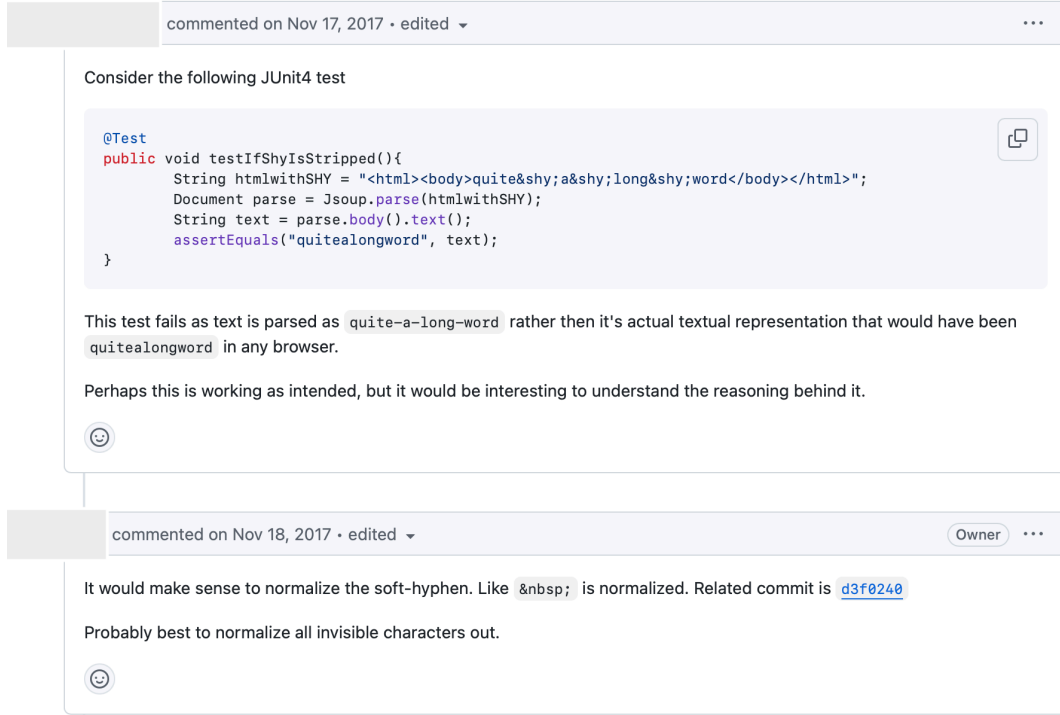


Fig. 8. An issue reported requiring developers to resolve; however, after fixing it, another bug is inadvertently introduced. See the original issue at: <https://github.com/jhy/jsoup/issues/978>

```
@@ -131,6 +131,11 @@
+ public static boolean isInvisibleChar(int c) {
+     return Character.getType(c) == 16
+         && (c == 8203 || c == 8204 || c == 8205 || c == 173);
+ }
@@ -160,7 +165,7 @@
    accum.append(' ');
    lastWasWhite = true;
}
- else {
+ else if (!isInvisibleChar(c)) {
    accum.appendCodePoint(c);
    lastWasWhite = false;
    reachedNonWhite = true;
}
```

Fig. 9. Regression-inducing changes where the intention was to fix an issue, but an unintended regression was introduced

Bug-inducing changes – This regression bug occurred when developers attempted to fix an issue raised by the user as shown in Figure 8. In this issue, users pointed out that the current version requires an enhancement of text rendering and formatting features as the `­` character was displaying as a hyphen (-). To resolve this, developers constructed the `isInvisibleChar` method to normalize text by omitting invisible characters, including *zero-width space* (ZWSP),

zero-width non-joiner (*ZWNJ*), *zero-width joiner* (*ZWJ*), and *soft hyphen* (*SHY*), as illustrated in Figure 9. Unfortunately, removing the *ZWNJ* and *ZWJ* caused a change in the emoji’s length after applying this function; an unintended behavior that was only identified in other issues after several years (see Figure 10).

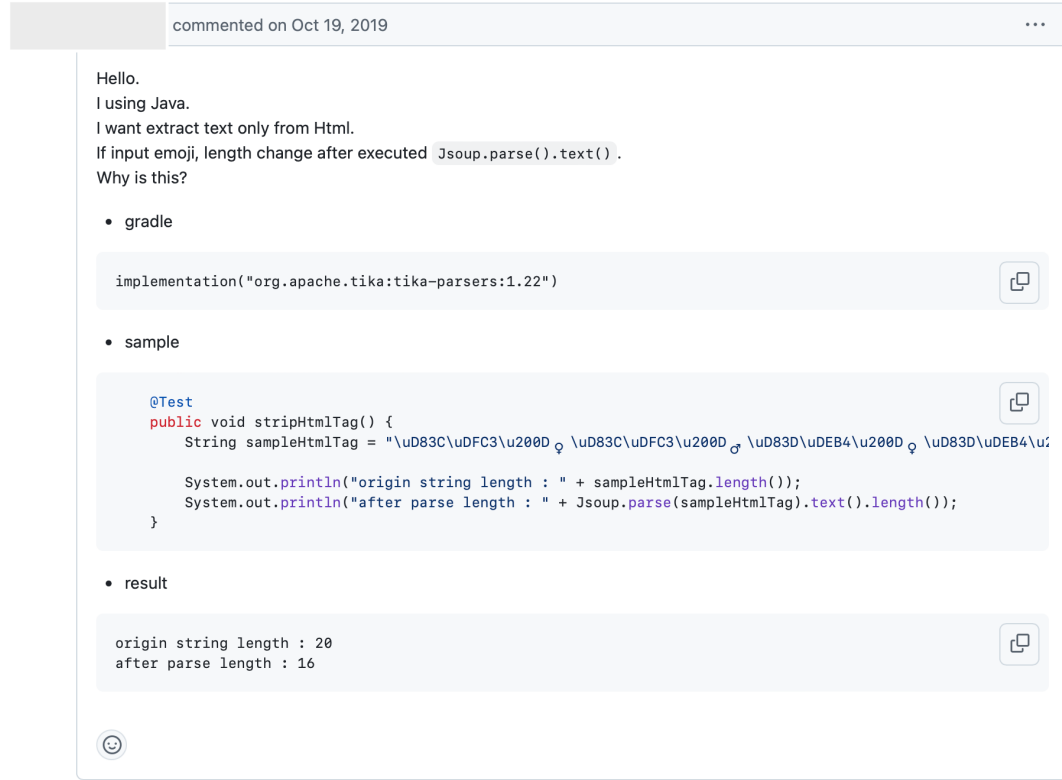


Fig. 10. An issue reported requiring developers to resolve. This issue is caused by the fix of a problem introduced two years prior (see Figure 8). See the original issue at: <https://github.com/jhy/jsoup/issues/1269>.

```
@@ -132,8 +132,8 @@
    public static boolean isInvisibleChar(int c) {
-        return Character.getType(c) == 16
+        && (c == 8203 || c == 8204 || c == 8205 || c == 173);
+        return c == 8203 || c == 173;
    }
}
```

Fig. 11. Bug-fixing changes made by developers to resolve the regression bug.

Bug-fixing changes - As illustrated in Figure 11, to fix the regression bug, developers modified the `isInvisibleChar` method to selectively only drop the characters requiring removal, specifically *SHY* and *ZWSP*, while preserving functional invisible characters such as *ZWJ* and *ZWNJ*. Although this fix is relatively straightforward, it necessitates a deeper understanding of the underlying bugs and the initial purpose of bug-inducing changes. The root cause of the issue lies in the overly broad handling of invisible characters in the `isInvisibleChar` method. The original intent of the text normalizer was to remove unnecessary formatting characters (e.g., *ZWSP* and *SHY*) to facilitate cleaner text

Manuscript submitted to ACM

processing. However, the approach unexpectedly removed $ZW\mathcal{J}$ and $ZWN\mathcal{J}$ characters, which are essential for joining characters into meaningful sequences. In particular, in contexts such as emojis or in scripts such as Arabic. The bug fix, therefore, needs to ensure that $ZW\mathcal{J}$ and $ZWN\mathcal{J}$ are retained, while still allowing for the removal of $ZWSP$ and SHY .

We can see that the generation of this fix required more than simply adjusting the buggy version to pass the test cases. For instance, reverting the code to the working commit would indeed pass the test case, but it would also undo the feature improvements made by the developers. Thus, careful consideration was needed to ensure the changes did not negatively impact other functions. This was accomplished by examining the bug-inducing changes and comparing the working version before the modifications with the failures introduced afterwards.

2.3.3 Problem Formulation. Formally, each regression bug instance in our benchmark consists of the following components: (i) a buggy program version P_b ; (ii) a test suite T , including at least one failing regression-witnessing test case; and (iii) a bug-inducing commit, from which the bug-inducing changes are extracted and represented as a diff file. The goal of the repair task is to generate a patch p such that the patched program $P_b + p$ passes all tests in T . The bug-inducing commit is the key distinguishing element of our benchmark compared to general-purpose APR benchmarks, as it provides historical context about how the regression was introduced, which is not available in traditional bug repair settings.

In our setting, the bug-inducing commit is assumed to be already identified, for example via tools such as *git bisect* or *SZZ* [61]. This assumption allows us to isolate and evaluate the impact of bug-inducing change information on repair effectiveness, without conflating it with the orthogonal challenge of commit identification. A similar assumption is commonly made in APR research with respect to fault localization. Specifically, by assuming perfect fault localization, existing techniques isolate repair capability from localization capability, enabling a more focused evaluation of patch generation [24, 39, 60, 72]. In the same spirit, our assumption of a known bug-inducing commit allows us to study how regression-specific context influences the repair process in isolation.

3 Study Setup

In this section, we describe the setup of our empirical study. We begin by outlining the research questions and presenting the overall workflow of the study. We then provide details of the benchmark construction process. Following that, we describe the selection criteria and experimental settings of the automated program repair (APR) techniques under our evaluation. Finally, we present the evaluation metrics used to assess the effectiveness of the repair methods.

3.1 Research Design

Our study is guided by the following research questions:

RQ1. *What is the degree of quality and diversity of the REGMINER4APR benchmark for APR studies?* This research question aims to examine the overall quality and characteristics of the REGMINER4APR benchmark, with the goal of assessing its suitability for supporting research in APR.

RQ2. *How effective are existing APR techniques at repairing regression bugs?* This research question aims to investigate the effectiveness of traditional and advanced APR techniques in repairing regression bugs.

RQ3. *What is the impact of providing additional context, particularly bug-inducing change information, on APR techniques?* This research question aims to explore whether providing additional context, specifically bug-inducing change information, can enhance the repair performance.

RQ4. *What is the contribution of each contextual element in the bug-inducing change information to repair effectiveness?* This research question investigates the individual contribution of different components of the bug-inducing change information to repair effectiveness through an ablation study.

Our study begins with the construction of the REGRESSIONBUG4APR benchmark, which includes 200 regression bugs collected from widely used real-world Java and Python GitHub repositories. Since data availability has long posed a challenge for APR research [27, 28, 44, 68], existing regression bug datasets are outdated and difficult to reproduce. In addressing **RQ1 (Section 4)**, we analyze our benchmark construction process and conduct an in-depth analysis of human-written patches to demonstrate the quality and diversity of the dataset. Building on this high-quality benchmark, we address **RQ2 (Section 5)** by empirically evaluating several APR techniques, including both traditional and advanced LLM-based approaches, on their ability to repair regression bugs. These evaluations are performed without considering the regression-inducing context, aligning with the conventional APR workflow, that typically take as input the buggy program and its associated test suite. In addressing **RQ3 (Section 6)**, we focus on prompt-based APR techniques, motivated by their demonstrated effectiveness, inherent flexibility in model usage and prompt construction, and greater data efficiency compared to fine-tuning-based approaches. We investigate whether incorporating regression-specific context, specifically bug-inducing change information, into the prompt design can enhance repair effectiveness. Furthermore, we conduct a qualitative analysis of plausible patches to examine how models leverage this contextual information to repair regression bugs. Finally, in addressing **RQ4 (Section 7)**, we build on the findings of RQ3 and conduct an ablation study on the best-performing configuration, conversational ChatGPT-4o, to isolate and evaluate the individual contribution of each contextual element within the bug-inducing change information. This allows us to determine which components are most informative for guiding LLM-based regression repair and to better understand the sources of improvement observed in RQ3.

3.2 Benchmark Construction

In this section, we outline the process for constructing a benchmark of regression bugs. Figure 12 illustrates an overview of the benchmark construction.

3.2.1 Project Selection. In this study, we focus on Java and Python regression bugs. Both languages are among the most widely used programming languages today, as consistently ranked in the TIOBE Index⁸, with Python ranked first and Java ranked fourth at the time of this study. While there has been a substantial body of research on regression errors in C/C++ projects [13, 51, 66, 81, 83], most of it dates back several years, leaving a gap in up-to-date studies on Java and Python regression errors.

Java. We selected well-known libraries and tools hosted on GitHub, implemented in Java, that meet the following criteria: (i) The project has more than 500 commits, used as a proxy to ensure sufficient development activity and code evolution; (ii) The project uses Maven or Gradle as the build system and JUnit test cases, ensuring compatibility with RegMiner tool and test execution environment; (iii) The project is not related to mobile or distributed platforms (e.g., Android), which often require specialized environments (e.g., emulators or network configurations) that complicate reproducibility; and (iv) The project is not structured as a multi-module build, as multi-module builds introduce complexity in dependency resolution and test isolation, which can hinder accurate mining, compiling, and test execution during regression analysis. Following these criteria, we sampled and selected 79 Java repositories⁹ hosted on GitHub.

⁸ Available at: <https://www.tiobe.com/tiobe-index/>

⁹ Available at: https://github.com/brojackvn/RegMiner4APR-Framework/blob/main/GITHUB_REPO_MINING.MD

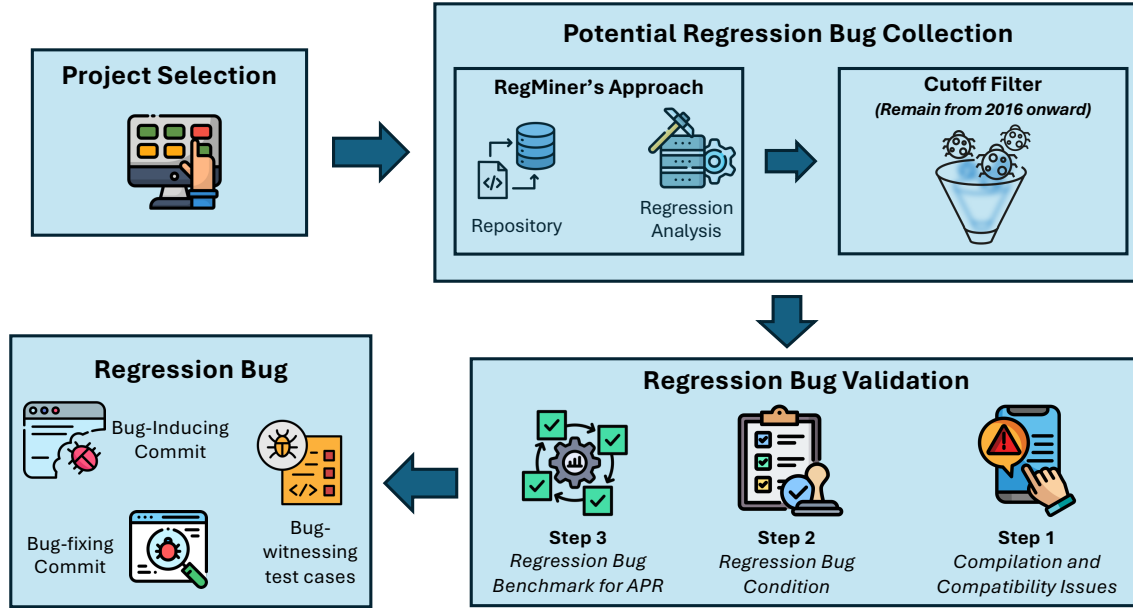


Fig. 12. Overview of the REGRESSIONBUG4APR benchmark construction.

This sample size was chosen due to the time-intensive nature of the bug mining tool, REGMINER, as well as the manual effort required for subsequent steps such as patch minimization and bug analysis.

Python. We selected two widely used open-source projects: *pandas*¹⁰, a widely adopted data analysis and manipulation library, and *Django*¹¹, a high-level web framework. These projects were selected because they represent diverse application domains and, among the Python projects we evaluated, they provided the highest number of successfully reproducible commit histories. Due to environment reproduction challenges specific to Python projects, we explored only the feasible portion of each project’s commit history to identify regression bugs. Specifically, for *pandas*, we examined commits from b89f1d0d05 (June 3, 2024) to b015a3bfb4 (September 12, 2025) on the main branch; for *Django*, we examined commits from 306607d5b9 (September 16, 2021) to b89f1d0d05 (January 10, 2026) on the main branch.

3.2.2 Potential Regression Bug Collection. For Java, we leverage RegMiner [64], a well-known regression mining tool, to identify potential regression bugs from the selected GitHub repositories. In total, we identified 1,631 potential regression bug candidates from these repositories. To ensure that our dataset is both timely and up-to-date, we filtered this set to retain only those whose bug-fixing commits were introduced from 2016 onward. We chose this cutoff because the most recent existing dataset on Java regression errors, CIBugs [28], primarily includes bugs from before 2017, and we aim to provide a more up-to-date benchmark. After applying this filter, we retained 758 potential regression bugs for further validation.

For Python, we developed a validation tool¹² that follows the same spirit as RegMiner. For simplicity, certain optimizations for minimizing search overhead, such as the heuristic algorithm used in RegMiner, are not implemented

¹⁰Available at: <https://pandas.pydata.org>

¹¹Available at: <https://github.com/django/django>

¹²Available at: <https://github.com/brojackvn/PyRegression-Mining-Tool>

due to the complexity of porting those components from Java to Python. Specifically, the tool identifies regression bugs by migrating test cases across commit histories to detect regressions within a reproducible execution environment. Due to the complexity of Python environment reproduction, we focus on a curated list of reproducible commits rather than exhaustively searching the full commit history. A validation step is applied at the end of the process to confirm that each identified instance satisfies the regression bug conditions described in Section 3.2.3. Through this process, we identified 50 confirmed Python regression bugs.

3.2.3 Regression Bug Validation. However, it is possible that not all extracted candidates represent true regressions since RegMiner employs heuristics in its mining process. Therefore, after collecting the set of potential regression bugs, we developed an automated validation tool to identify true regression bugs. This tool applies a series of filtering criteria to evaluate the relevance and accuracy of each case as follows:

- *Step 1: Compilation and Compatibility Issues (Executability).* This step is necessary because RegMiner uses test cases extracted from the bug-fixing commit, which need to be migrated to earlier versions using rule-based heuristics. Our validation tool takes as input the potential bug-inducing commit, bug-fixing commit, and the corresponding test case that exposes the regression. It then attempts to reconstruct the four relevant commit versions with migrated test cases. However, 236 of the 758 potential regression bugs encountered failures during this step, falling into two main categories:
 - *Compilation failures*, where the migrated test case cannot be compiled against earlier program versions due to structural incompatibilities introduced between commits. For example, in the `w3c/epubcheck` project (bug-inducing commit `1268a8fb`, bug-fixing commit `356fac0f`, test case `com.adobe.epubcheck.ops.OPSCheckerTest#testCustomElements`), the bug-fixing commit refactored `OPSCheckerTest` by introducing new method signatures, renaming symbols, and adding the `Messages` class, none of which exist in the earlier bug-inducing commit. As a result, the migrated test file references symbols and method overloads that cannot be resolved against the older codebase, causing `maven-compiler-plugin` to fail with `cannot find symbol` and `ambiguous method reference` errors.
 - *Dependency compatibility failures*, where required dependencies are no longer available in remote repositories (e.g., Maven Central), preventing the project from being built. In some cases, dependencies are removed within days of the bug being mined. Even if the project compiled successfully at the time of mining, subsequent reconstruction attempts may fail with errors such as: “[ERROR] Caused by: The following artifacts could not be resolved: ...”. This highlights the inherent instability of relying on remote dependency repositories for historical version reconstruction, and directly motivates our design decision in Section 3.2.4 to clone each required commit version along with all necessary dependencies, rather than relying on remote repositories at runtime. After filtering out these cases, 522 potential regression bugs remained.
- *Step 2: Regression Bug Conditions (Validity).* This step ensures the behavioral conditions of a true regression bug are satisfied, as RegMiner may yield false positives. Specifically, our validation tool verifies that the test case passes on both the bug-fixing commit and the version prior to the bug-inducing commit, and fails on both the bug-inducing commit and the version prior to the bug-fixing commit. Bugs that do not satisfy this condition are discarded. After this step, 264 potential regression bugs remained.
- *Step 3: Regression Bug Benchmark for APR (Utility).* This step ensures that the regression bugs included in the benchmark are suitable for automated program repair (APR). We integrated an additional condition into our validation tool to check that the buggy version fails only the test cases that exposes the regression, and that

the bug-fixing commit passes all test cases. Although this may exclude some complex but valid regressions (e.g., those affecting multiple test cases beyond the ones exposing the regression), it helps construct a clean and regression-focused benchmark tailored for APR research.

Ultimately, through this process, we curated a dataset of 150 confirmed Java regression bugs, drawn from 79 distinct GitHub repositories. Combined with 50 confirmed Python regression bugs, this yields the `REGRESSIONBUG4APR` benchmark comprising a total of 200 regression bugs.

3.2.4 Data Storage. To ensure extensibility and durable reproducibility, we store the validated regression bugs in a structured database hosted on GitHub. This centralized storage facilitates easy access and management of the regression bug dataset. In addition, we have developed an interface on top of this database to streamline usage for researchers, support reproducible studies in program repair, and enable future extensions without requiring environment-specific setup for each individual bug. The key characteristics of our storage design include:

- *Durable reproducibility:* To guard against future dependency issues (e.g., broken builds due to removed or updated dependencies), we clone each required commit version along with all necessary resources and dependencies. For Maven-based projects, we use the built-in command `mvn dependency:copy-dependencies`; for Gradle-based projects, we implemented a custom function to extract and preserve all required dependencies.
- *Extensibility:* The database is structured to support future expansions of the benchmark. Each regression bug is represented by four key snapshots: the *bug-inducing* commit and its immediate predecessor, and the *bug-fixing* commit and its immediate predecessor. Each regression instance is stored within a branch in our GitHub repository, allowing new cases to be added easily.
- *Easy of Use:* To support ongoing research, we integrated the benchmark into a framework that provides essential commands to interact with the benchmark without extensive configuration. This framework offers uniform access to build and execution tasks by abstracting the underlying build systems. Supported commands include `info`, `env`, `checkout`, `compile`, and `test`.

At this stage, we finalize the construction and storage of the `REGRESSIONBUG4APR` benchmark. To enable further analysis and better support APR studies, we extract both the bug-fixing changes (i.e., human-written patches) and the bug-inducing changes.

3.2.5 Bug-inducing and Bug-fixing Change Extraction and Minimization. Since developers may introduce unrelated edits alongside bug fixes, isolating the critical changes associated with each regression error is essential for accurate benchmark analysis and for supporting APR studies (e.g., patch validation). To achieve this, we extract both the bug-inducing and bug-fixing changes by performing the following two steps, which aim to best capture the modifications relevant to each regression error.

Step 1: Test Execution and Coverage Collection. We begin by executing the bug-witnessing test cases on both the bug-inducing and bug-fixing commits to collect line-level code coverage. By comparing the coverage data with the corresponding code changes, we discard any lines not covered by the test cases and retain only those directly involved in triggering or fixing the bug.

Specifically, to isolate the bug-inducing and bug-fixing changes, we first collect the code coverage data from the relevant commits by executing the bug-witnessing test cases using JaCoCo¹³, an industry-grade Java code coverage

¹³Available at: <https://www.jacoco.org/jacoco/>

library, for Java bugs, and Coverage.py¹⁴ for Python bugs. The coverage data is collected by running the targeted test cases with the respective coverage tool, which produces a coverage report containing the raw line-level coverage information for the executed classes and methods.

From the resulting coverage report, we extract the set of lines covered by test cases. The overall coverage information is represented as: $\mathcal{G} = \{\text{file}_i \mapsto \{\text{cov}_{i1}, \text{cov}_{i2}, \dots\} \mid i = 1, \dots, n\}$, where n is the number of files covered by the test cases, and cov_{ij} denotes a line number covered in file_i . We then use a diff tool to obtain the code changes in both the bug-inducing and bug-fixing commits. These changes are computed as diffs against each commit's immediate predecessor. The code changes are represented as: $\mathcal{H} = \{\text{file}_i \mapsto \{\text{chg}_{i1}, \text{chg}_{i2}, \dots\} \mid i = 1, \dots, m\}$, where m is the number of files modified, and chg_{ij} denotes a line number modified in file_i . Finally, for each file_i , the set of relevant changes covered by the test cases is computed as the intersection: $\mathcal{R}_i = \mathcal{G}(\text{file}_i) \cap \mathcal{H}(\text{file}_i)$, where $\mathcal{R}_i$ denotes the set of changed lines in file_i that are covered by the targeted test cases. Each commit, bug-inducing or bug-fixing, has its own computed set $\mathcal{R}_i$, representing the code changes directly exercised by the bug-witnessing test cases. These refined sets are crucial for pinpointing the specific code modifications responsible for the observed regression behavior. For specific types of changes, however, we apply the following additional considerations:

When computing $\mathcal{R}_i$, we apply the following additional considerations based on the type of change:

- *Bug-inducing changes.* We focus on identifying lines that may cause the bug. If a changed branch of an if/else statement is not covered by any test case, we remove it, as it cannot be responsible for the observed failure.
- *Bug-fixing changes.* Uncovered lines require more careful consideration. Even if a line is not directly exercised by the test cases, it may still constitute a semantically related component of the fix. For example, when test cases cover only one branch of an if/else statement, but the other branch is still part of the fix. To ensure correctness, we validate uncertain removals by reinserting them into the buggy version and running all unit tests. If reinserting a removed change fixes the bug, we conclude that the change is bug-fixing and retain it. Otherwise, it can be safely discarded. This validation step relies on test execution rather than human judgment.

Step 2: Patch Minimization through Rules. To further enhance the clarity and effectiveness of our patch analysis, we manually identify and remove code changes that do not affect the bug-inducing or bug-fixing behavior. Following Defects4J¹⁵, we define the following rules to guide our patch minimization efforts:

- *Code Formatting:* Code changes that alter only the appearance of the code without affecting its behavior may be excluded. This includes comments, unnecessary new lines, and whitespace or indentation changes.
- *Simple Code Refactoring:* Code changes that modify the code structure syntactically without introducing any behavioral difference are removed. For example, in *RegressionBug-106*, we removed a change from `if (!excludePrivate || !foundTld.isPrivate)` to `if (!(excludePrivate && foundTld.isPrivate))`. Although these two expressions are logically equivalent, the change affects only the syntactic structure of the condition and is therefore excluded as a refactoring change unrelated to the bug.

While this step involves human intervention and may introduce subjective bias, we took two measures to mitigate this risk. First, we applied pre-defined rules to guide and constrain minimization decisions, reducing ambiguity and ensuring consistency across all bug instances. Second, all manual minimization decisions were independently reviewed by a second author, and any disagreements were resolved through discussion.

¹⁴Available at: <https://coverage.readthedocs.io>

¹⁵Available at: <https://github.com/rjust/defects4j/blob/master/framework/bug-mining/Patch-Minimization-Guide.md>

After applying both steps, the overall patch size decreased from 7,368 to 3,385 lines after Step 1, and further to 2,853 lines after Step 2 for human patches, representing an overall reduction of approximately 61.28%. For bug-inducing changes, the size decreased from 45,995 to 14,045 lines after Step 1, and further to 13,334 lines after Step 2, representing an overall reduction of approximately 71.01%.

3.3 APR Technique Selection and Experimental Settings

In this section, we present the APR techniques and their experimental settings evaluated in our empirical study. We revisit both traditional and LLM-based approaches to assess their applicability to regression bugs.

3.3.1 Traditional APR Techniques. To examine whether traditional search-based and pattern-based approaches, utilizing mutation operators and fix patterns, can effectively repair regression errors, we selected a set of widely studied heuristic-based APR tools. Specifically, we include five search-based techniques: GenProg [36], Kali [55], Cardumen [46], Arja [82], and jMutRepair [45], following the selection made by Kabadi et al. [28]. For template-based APR, we include TBar [39], which systematically aggregates fixing patterns derived from prior template-based APR techniques and is widely regarded as a representative approach in the program repair literature. We do not evaluate semantic-based APR approaches in our study as they often require the inclusion of manually-written models for system calls whose source codes are not available.

To conduct our experiments, we utilize Cerberus, a unified framework for running and evaluating traditional APR tools, which has been shown to reproduce identical results as originally reported by the respective tool authors on Defects4J, confirming the reliability of our experimental setup [59]. All experiments for these tools are performed on the server equipped with an AMD CPU (16 cores) and 64GB of RAM. We retain the default configuration settings provided by Cerberus for each tool to ensure consistency, reproducibility, and tool-agnostic comparisons [39, 45]. While some parameters could be tuned, such as the maximum number of generations and population size for genetic-programming-based approaches (e.g., jGenProg, jKali, jMutRepair, Cardumen, and Arja), as well as the set of predefined repair patterns for TBar, aggressive per-tool tuning may introduce bias and reduce comparability across tools. We also note that the default configuration for genetic-programming-based tools already uses a maximum of 1,000,000 generations and an overall timeout of 30 minutes, a budget that aligns with developers' expectations for APR tools in practice [18, 52].

3.3.2 Fine-tuning-based APR Techniques. To investigate how well fine-tuning-based approaches perform when applied specifically to regression errors, we revisit several recently proposed fine-tuning-based APR methods. These techniques have demonstrated promising results compared to earlier deep learning-based methods [24]. Following the work of Jiang et al. [24], we include several full-parameter fine-tuned models that exhibit significant performance improvements over their original pretrained counterparts. Specifically, we evaluate InCoder-1B and InCoder-6B [20], as well as CodeGen-2B and CodeGen-6B [50]. In addition, we evaluate RepairLLaMA [60], a fine-tuned version of CodeLLaMA [56] trained using QLoRA [16], a parameter-efficient fine-tuning method. This technique is particularly interesting due to its reduced memory and computation requirements during training while still achieving strong repair performance.

For input and output representation, we adopt model-specific configurations aligned with the respective fine-tuning procedures, as illustrated in Figure 13 and Figure 14. In particular, *InCoder* and *CodeGen* [24] require the entire buggy function along with annotations marking the buggy location. The buggy segment is highlighted using the markers `"/ buggy lines start"` and `"/ buggy lines end"`, followed by a placeholder line starting with `"/ fixed lines:"`, which designates where the fix should be generated. The model is expected to output the corresponding fixed code chunk that replaces the buggy code. In contrast, *RepairLLaMA* [60] adopts a different input format tailored to its infilling-based approach.

Input and Output Representation of InCoder and CodeGen	Input and Output Representation of RepairLLaMA
<pre>private static boolean isAssignableFrom(Class<?> target, KType source) { Type parameterType = ReflectJvmmapping.getJavaType(source); // buggy lines start if (parameterType instanceof Class) { return target.isAssignableFrom((Class<?>) parameterType); } return false; // buggy lines end } // fixed lines: return ResolvableType.forClass(target) .isAssignableFrom(ResolvableType.forType(parameterType));</pre>	<pre>private static boolean isAssignableFrom(Class<?> target, KType source) { Type parameterType = ReflectJvmmapping.getJavaType(source); // buggy code // if (parameterType instanceof Class) { // return target.isAssignableFrom((Class<?>) parameterType); // } return false; // return false; <FILL_ME> } return ResolvableType.forClass(target) .isAssignableFrom(ResolvableType.forType(parameterType));</pre>

Fig. 13. The input and output representation of InCoder and CodeGen.

Fig. 14. The input and output representation of RepairLLaMA.

The input consists of a code prefix followed by a comment beginning with the marker `"/ buggy code"`, which introduces the buggy lines retained in commented-out form. This is followed by the special token `<FILL_ME>`, which indicates the location where the model should generate the fix. The model is expected to output the corresponding fixed code chunk that replaces the buggy segment while preserving the original context.

All experiments are conducted using FLAMES [33], a unified framework for executing and configuring APR evaluations with large language models. We run all fine-tuning-based techniques on our highest-performance machine, which is equipped with a single NVIDIA A100 GPU (80GB VRAM), 250GB of system RAM, and a 32-core Intel Xeon CPU running at 2.90GHz. To maintain consistent generation quality across models, we set the temperature hyperparameter to 1.0 to control output diversity. Additionally, to prevent out-of-memory crashes on our setup, we limit the number of generated candidates using a beam size of 10.

3.3.3 Prompt-based APR Techniques. To investigate how well prompt-based approaches perform when applied specifically to regression errors, we explore techniques that leverage general-purpose large language models (LLMs), such as ChatGPT, without requiring any fine-tuning. These approaches rely solely on prompt engineering to guide the model in generating plausible patches. In this category, two common strategies have emerged: *zero-shot prompting APR* and *conversational APR*. Zero-shot prompting APR involves issuing a single prompt to the LLM to generate a patch, without relying on any interactive feedback or prior conversational context [73]. In contrast, conversational APR techniques incorporate multiple dimensions of feedback to iteratively query the model. These methods maintain a conversation history and leverage prior failed patching attempts to improve future generations through prompting [72, 80].

In our experiments, we select advanced models from the ChatGPT family to implement prompt-based repair. Following prior work by Xu et al. [76], we note that the next-token prediction objective used by decoder-only LLMs (e.g., GPT-4) is misaligned with the masked span prediction objective employed in current infilling-style methods. Since these models are used without any fine-tuning, the effectiveness of LLMs can be improved by aligning the repair task with their training objective; that is, allowing them to refine the entire program without explicitly providing buggy code hunks.

To this end, we input the entire buggy method along with relevant debugging information as part of the prompt. Following prior works [72, 73, 80], we manually examined a few alternative approaches using selected bugs through the web-based interface of ChatGPT¹⁶. Based on these insights, we construct our prompt as illustrated in Figure 15. More specifically, we initialize ChatGPT with the system instruction *"You are an Automated Program Repair Tool"* to explicitly

¹⁶ Available at: <https://chatgpt.com>

define its role. We then construct the user prompt by providing various types of contextual information related to the buggy function. This includes the full body of the buggy function, followed by a list of failing test cases along with their associated error messages. Initially, only functional errors are present in the benchmark. In this case, our framework REGRESSIONBUG4APR supports the extraction of key diagnostic information, including the name of the failing test case (which often serves as a concise summary of the function under test), the error type (e.g., *org.junit.ComparisonFailure*), and the detailed error message (e.g., *expected:<[6]> but was:<[1]>*), which provides concrete evidence of the failure. Finally, the prompt concludes with a chain-of-thought indicator, "Let's think step by step to fix the bug", followed by the task description, "Please provide a correct function". This prompt structure encourages the model to reason about the bug and generate a complete and syntactically correct replacement for the entire function, rather than returning an unformatted or incomplete patch.

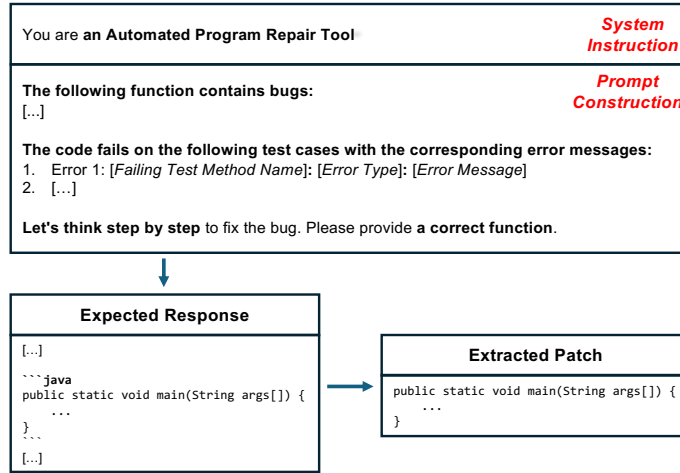


Fig. 15. Structure of our prompt-based APR query. The code snippet and error messages are omitted for clarity. The expected response is formatted to allow reliable extraction of the repaired function from the LLM's output.

Table 1. Prompt Templates for Feedback Cases

Case	Prompt Template
Compilation Error	The fixed version is not compilable. The code has the following compilation error: {error_message} Please provide the correct function along with any required imports.
Functional Error	The fixed version is still not correct. The code fails on the following test cases with the following error messages: {error_message} Please provide the correct function again.
No Response Code	The response does not provide the code function. Please provide the correct function again.
Timeout	The fixed version is still not correct and ran out of time. Please provide the correct function again.

In the case of zero-shot prompting APR, we use only the initial prompt to query ChatGPT and extract a candidate patch from the model's output. In contrast, conversational APR techniques begin with the same initial prompt but

continue with an interleaved process of patch generation and validation-driven feedback. Each generated patch is immediately validated by compiling and running the program against the test suite. If the patch fails, we construct a feedback prompt that includes both the previously generated incorrect patch and the associated failure information, which is then used to prompt the next generation. We observe that there are four main cases in which feedback is provided to the model: (i) **Compilation Error**, when the patch cannot be compiled; (ii) **Functional Error**, when the patch compiles but fails one or more test cases; (iii) **No Response Code**, when the model does not return a valid code function; and (iv) **Timeout**, when the patch execution exceeds a predefined time limit, often indicating an infinite loop or inefficient implementation. For each case, we formulate a corresponding natural language feedback prompt that communicates the type of failure and requests a corrected function. These feedback templates are summarized in Table 1.

The repair tool is developed in Python and communicates with the ChatGPT API endpoint¹⁷. We use *gpt-3.5-turbo-0125* and *gpt-4o-2024-08-06*, which were the most up-to-date and cost-effective models at the time of implementation. Inference and evaluation scripts are executed on a server equipped with a 16-core AMD CPU and 64GB of RAM. To control a moderated randomness of text generation, we follow prior work and set the *temperature* hyperparameter to 1.0, which is also the default value provided by OpenAI. Additionally, we adopt a relatively small sampling size of 10 candidates per bug, given the significantly higher inference cost (in terms of both time and financial budget) of LLMs compared to other APR tools. This setting also ensures a fair comparison with the fine-tuning-based approaches evaluated in this study.

For the conversational strategy, since ChatGPT has a limited context window¹⁸, meaning it cannot process arbitrarily long input sequences, we introduce a threshold on conversation length (i.e., the number of prompt-response exchanges in a continuous repair session). Once this maximum conversation length is reached, the repair process is restarted from the initial prompt. A maximum conversation length of 1 corresponds to zero-shot prompting APR. As the maximum conversation length increases, the model can incorporate more repair history (e.g., previous patches and corresponding feedback) into its prompts. In our experiments, we retain the sampling size of 10 and set the maximum conversation length to 5. This setting still ensures a fair comparison, as it assumes that each new prompt in the conversation builds upon the model’s previous patch rather than generating a completely new patch.

3.3.4 Prompt-based APR Techniques Augmented with Bug-Inducing Change Information. To explore the potential of leveraging regression bug information in automated program repair, we focus on prompt-based techniques, which are motivated by their effectiveness, inherent flexibility in model usage and prompt construction, and greater data efficiency compared to fine-tuning-based approaches. In particular, large language models, such as ChatGPT, have demonstrated remarkable generalization capabilities and an ability to understand and generate code from natural language instructions [41]. These strengths make LLMs especially well-suited for our investigation. We hypothesize that providing LLMs with additional contextual information, specifically bug-inducing change information, can significantly enhance their ability to fix regression bugs. Identifying where and how a regression was introduced offers valuable insight that can support the patch generation process.

In our experiments, we extend existing prompt-based APR frameworks by adapting the prompt design to explicitly incorporate regression-specific context. While our implementation maintains the two repair strategies, *zero-shot prompting* and *conversational APR*, we design the prompt to align with our research objective of investigating how

¹⁷ Available at: <https://platform.openai.com/docs/overview>

¹⁸ Available at: <https://platform.openai.com/docs/guides/chat>

augmenting prompts with bug-inducing change information affects repair effectiveness. The overall interaction structure, inference process, and configuration settings remain consistent with the prompt-based APR techniques described earlier in this section.

You are an Automated Program Repair Tool System Instruction The following function contains bugs: [...] Prompt Construction Regression-Inducing Changes: [...] Regression-inducing commit message: [...] The code fails on the following test cases with the corresponding error messages: 1. Error 1: [Failing Test Method Name]: [Error Type]: [Error Message] 2. [...] Please think step by step to identify the root cause from the regression-inducing change information in order to fix the regression bug. Then, provide a correct version of the function.	You are an Automated Program Repair Tool System Instruction The following function contains bugs: [...] Prompt Construction Regression-Inducing Changes: There are no changes in the buggy function at regression-inducing commit. The bug is caused by changes made elsewhere in the same commit. Focus on the bug-inducing commit message. Regression-inducing commit message: [...] The code fails on the following test cases with the corresponding error messages: 1. Error 1: [Failing Test Method Name]: [Error Type]: [Error Message] 2. [...] Please think step by step to identify the root cause from the regression-inducing change information in order to fix the regression bug. Then, provide a correct version of the function.
(a) Regression-inducing commit includes changes to the buggy function.	(b) Buggy function is unchanged, and the fault is introduced by other changes in the regression-inducing commit.

Fig. 16. Prompt templates designed for prompt-based APR with regression-inducing change information.

The primary distinction in this approach lies in the design of the prompt. Figure 16 illustrates two prompt templates used in prompt-based APR for regression bugs, each corresponding to a different scenario based on how the regression was introduced. Specifically, we initialize ChatGPT with the system instruction "You are an Automated Program Repair Tool" to clearly define its role. The user prompt is then constructed by providing contextual information related to the buggy function, including its full body and a list of failing test cases along with their corresponding error messages. At the initial stage, our benchmark includes only functional errors. To support this process, we use REGRESSIONBUG4APR, which extracts diagnostic information from the failing test cases, similar to the prompt-based APR setup described earlier. In the case where the regression-inducing commit includes changes to the buggy function, we extract the corresponding code diff and commit message to provide additional context, as shown in Figure (16a). This additional information helps guide the model toward understanding how the changes may have introduced the regression. In contrast, some regression bugs are caused by changes made elsewhere in the same regression-inducing commit, leaving the buggy function itself unchanged. In such cases, as illustrated in Figure (16b), we explicitly note that the buggy function was not modified and highlight the commit message as a key signal for localizing the root cause of the errors. Finally, each prompt concludes with a specialized chain-of-thought instruction tailored for regression scenarios: "Let's think step by step to identify the root cause from the regression-inducing change information in order to fix the regression bug", followed by the task instruction: "Then, provide a correct version of the function". This structure encourages the model to reason about the root cause of the regression and generate a complete and syntactically correct replacement for the function, rather than producing an unstructured or partial patch.

3.4 Evaluation Metrics

Following prior work, we adopt two widely used metrics for evaluating APR tools: (i) the number of correct patches and (ii) the number of plausible patches. A plausible patch is one that passes all test cases, while a correct patch is semantically

or syntactically equivalent to the developer’s reference patch. We follow common practice in APR and manually assess semantic equivalence to determine correct patches.

Specifically, we compute three derived metrics: plausible rate (PR), correct rate (CR), and precision (P). The correct rate and plausible rate indicate the overall effectiveness of APR tools, while precision reflects the reliability of the generated patches with respect to the overfitting problem [32, 62].

$$PR = \frac{\# \text{Plausible Patches}}{\# \text{Bugs in Dataset}} \quad CR = \frac{\# \text{Correct Patches}}{\# \text{Bugs in Dataset}} \quad P = \frac{\# \text{Correct Patches}}{\# \text{Plausible Patches}}$$

4 RQ1: Quality and Diversity of the REGRESSIONBUG4APR Benchmark

To validate the quality and diversity of the REGRESSIONBUG4APR benchmark, we conduct an analysis of its construction process and perform a detailed study of its overall characteristics. Our methodology consists of two parts: assessing the quality of the benchmark construction and characterizing the features of the benchmark.

4.1 Benchmark Quality

We systematically analyze each filtering and validation step applied during the benchmark construction process, as detailed in Section 3.2. Note that this analysis covers the Java regression bug collection process only. The Java regression bugs are initially mined by RegMiner and subsequently re-validated using our own quality criteria to ensure they satisfy the conditions required for APR research. For Python, the validation process is integrated directly into the tool, such that any identified bug is already a confirmed regression bug suitable for APR research, without requiring additional validation steps, as described in Section 3.2.2. At each step, we record the number and proportion of potential regression bugs filtered out, as presented in Figure 17. This allows us to quantitatively assess the effectiveness of each criterion and demonstrate the overall reliability and soundness of the resulting REGRESSIONBUG4APR benchmark.

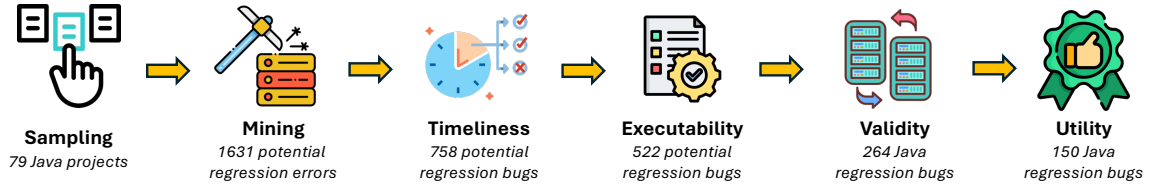


Fig. 17. Number of bugs filtered out during the Java regression bug validation process.

Overall, our validation process filters out 90.8% of the initially mined Java candidates, retaining only those that meet the criteria for true regression bugs. The resulting dataset not only preserves the defining behavioral characteristics of regression errors but is also specifically designed to support research on automated regression bug repair.

4.2 Benchmark Characterization

We characterize the features of REGRESSIONBUG4APR benchmark by conducting an in-depth analysis of its key properties. Specifically, we examine overall benchmark statistics and provide a detailed study of human-written patches, focusing on their associated repair operators.

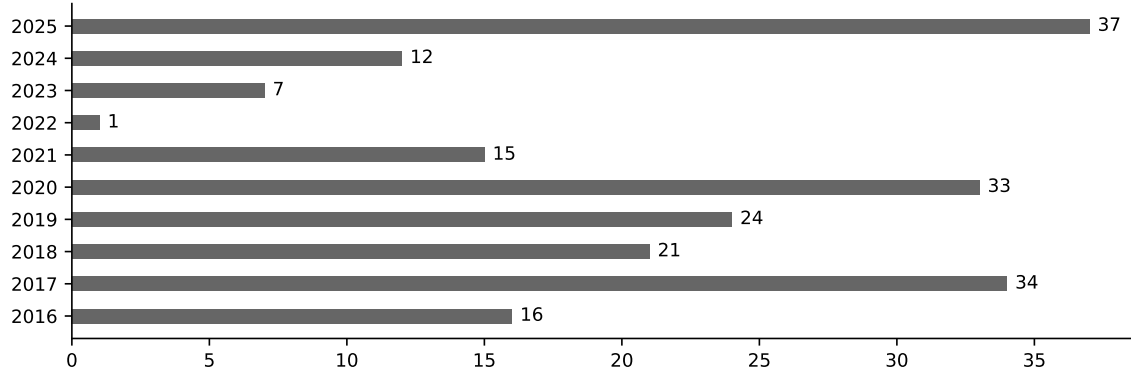


Fig. 18. Time distribution of the regression bugs in our benchmark

4.2.1 Subject Dataset: REGRESSIONBUG4APR. Our benchmark consists of 200 regression bugs extracted from widely used real-world Java and Python GitHub repositories. For each bug, REGRESSIONBUG4APR provides four snapshots of the project versions: the bug-fixing commit, the bug-inducing commit, and their immediate predecessors. Additionally, REGRESSIONBUG4APR bugs are characterized as follows: (i) they have been selected after 2016, as shown in Figure 18; (ii) they are related to source code fixes, with exclusions for changes within the build system, configuration files, documentation, or tests; (iii) they are reproducible, as each bug contains at least one test case that satisfies the conditions for regression errors mentioned in Section 3.2; and (iv) they are isolated, ensuring that bug-inducing changes and bug-fixing patches do not include unrelated modifications, such as new features or refactoring.

The detailed information about each bug are included in our artifacts, as provided in Section 1. Each bug in REGRESSIONBUG4APR is referenced using a simple notation: “*RegressionBug-*” followed by a bug ID ranging from 1 to 150 for Java regression bugs, and “*PyRegression-*” followed by a bug ID ranging from 1 to 50 for Python regression bugs.

4.2.2 Data Statistics. We begin by analyzing basic properties of patches in our benchmark. In particular, we focus on patch size and spreading scope, which help us understand the complexity and challenges of bug fixes.

Table 2. Descriptive statistics for patch size and scope

	Min	25%	50%	75%	90%	95%	Max
# Added lines	0	1	3	7	15	23	159
# Removed lines	0	0	1	2	8	16	111
# Patch size	0	2	4	10	22	42	226
# Chunks	1	1	1	3	6	12	93
# Modified files	1	1	1	1	2	3	24
# Modified classes	1	1	1	1	3	7	24
# Modified methods	0	1	1	1	3	5	66

Size of the RegressionBug4APR patches. This is presented by the number of lines added, removed and patch size, as shown in Table 2.

- *Added lines:* 25% of patches contain no added lines, while half of the patches add at most one lines. Patches with more than 15 added lines are less frequent.

- *Removed lines*: Half of the patches contain no removed lines, and 95% of the patches have no more than 16 removed lines.
- *Patch size*: The total size of a patch, calculated as the sum of added and removed lines, involves at most 4 lines for half of the patches. For 90% of patches, no more than 42 lines are affected.

Scope of the RegressionBug4APR patches. This is analyzed by the number of chunks, modified methods, modified classes, and modified files, as demonstrated in Table 2.

- *Chunks*: Defined as continuous lines of code changes, 75% of patches contain three or fewer chunks.
- *Modified methods*: 155 patches (77.5%) change one method.
- *Modified classes*: We observe that the number of modified classes is closely related to the number of modified files in each patch.
- *Modified files*: 26 patches require changes in at least two files to address the bug. Of these, 9 follow a unique regression-fixing pattern, namely *Revert to previous statement*.

Overall, the median patch size in REGRESSIONBUG4APR is four lines, with approximately 10% of patches affecting more than 22 lines. Among the 200 patches, 100 contain a single chunk, 155 modify only one method, and 174 are confined to a single file. The occurrence of larger patches is attributed to a specific regression bug-fixing pattern, namely *Revert to previous statement*, as highlighted in our bug analysis. This patch size distribution implies that the benchmark is suitable for evaluating a wide range of APR techniques. For instance, traditional APR tools typically produce small patches, while LLM-based approaches can handle larger modifications. Moreover, the presence of a few much larger patches highlights the need for future research into techniques capable of addressing more substantial repairs.

4.2.3 Bug Categorization. We further justify the diversity of the REGRESSIONBUG4APR benchmark by characterizing the constituent bugs. We dissect bugs in the benchmark into several different repair operators, of which some are obtained from Java defect classes proposed by Pan et al. [54] and Sobreira et al. [63]. Each operator is characterized by the type of code change performed to fix the defect. These operators correspond to foundational constructs that are common across imperative programming languages. Since Java and Python share the same core imperative constructs, such as assignments, conditionals, loops, method calls, and return statements and etc, we apply the same set of operators to annotate patches from both languages in our benchmark. Table 3 summarizes the groups of repair operators observed in the benchmark. We describe the details of each operator group below.

Table 3. Repair Operators

Acronym	Group	Action
asgn	Assignment	A/R/M
cnd	Condition	A/R/M
lp	Loop	A/R/M
mc	Method call	A/R/M
md	Method definition	A/R/M
obj	Object Instantiation	A/R/M
exp	Exception	A/R
ret	Return	A/R/M
var	Variable	A/R/M
rev	Revert to previous statement	N.A

abbreviation A: Add; R: Remove; M: Modify.

- *Assignment*: Changes involving a simple assignment operator (`=`), unary increment/decrement operators, or compound assignment operators with arithmetic operations. Actions applicable to this group include *addition* and *removal* at the statement level, and *modification* at the expression level (e.g., modifying the assigned value).
- *Condition*: Changes related to conditional branches, such as `if-else`, `switch-case`, and the ternary operator. Actions applicable to this group include *addition* and *removal* at the statement level, and *modification* at the expression level (e.g., modifying, expanding, or reducing the conditional expression).
- *Loop*: Changes related to loop constructions, such as `for` and `while`. Actions applicable to this group include *addition*, *removal* at statement level, and *modification* at both statement and expression levels.
- *Method Call*: Changes related to method calls. Actions applicable to this group include *addition* and *removal* at the statement level, and *modification* at the expression level (e.g., adding, removing, swapping, and modifying parameters).
- *Method definition*: Changes related to method definitions and signatures. Actions applicable to this group include *addition* and *removal* at the statement level, and *modification* at the expression level (e.g., adding or removing parameters, modifying parameter types, changing the return type, and altering the method's scope).
- *Object Instantiation*: Changes related to the instantiation of objects are observed through the keyword `new`. Actions applicable to this group include *addition* and *removal* at the statement level, and *modification* at the expression level (e.g., modifying the parameters passed in objects).
- *Exception*: A statement related to exception handling and throwing. Actions applicable to this group include *addition* and *removal* at the statement level (e.g., adding or removing `try-catch` or `throw` blocks).
- *Return*: Changes related to return statements. Actions applicable to this group include *addition* and *removal* at the statement level (e.g., wrapping with a conditional `if-else`), and *modification* at the expression level (e.g., modifying the logic expression of the return).
- *Variable*: Changes related to variable declaration and usage. Actions applicable to this group include *addition* and *removal* at the statement level, and *modification* at the expression level (e.g., modifying the variable type, replacing it with another variable or method call, and altering the assigned value).
- *Revert to previous statement*: Changes related to rolling back at the statement level. The rationale behind this action is that developers may have made mistakes that should have been left intact, resulting in the regression bug at hand. This is particularly unique for regression bugs, as it involves observing multiple versions and reasoning through the changes.

Following the regression categorization by Tan et al. [66], based on how the regression was introduced (as described in Section 2), our dataset contains 139 *local* bugs, 50 *remote* bugs, and 7 *unmask* bugs, with 4 bugs left uncategorized due to the complexity of their bug-inducing and bug-fixing changes. Figure 19 illustrates the overall distribution of repair operators for each type of regression bug. The radial axis corresponds to the repair operators discussed in the previous section, indicating their occurrence across the different bug types. Common patterns, such as *Conditional Addition*, *Method Call Addition*, *Method Call Modification*, *Assignment Modification* and *Assignment Addition*, are prevalent in all three types of regression bugs. Notably, the *Revert to Previous Statement* operator is more frequently observed in the *Local* bug category.

Figure 20 demonstrates the ranking of repair operators based on human patches, with the vertical axis representing the operators and the horizontal axis indicating the number of patches in which they are found. To enhance clarity, grey bars represent addition actions, diagonal striped bars indicate removal actions, white bars signify modification

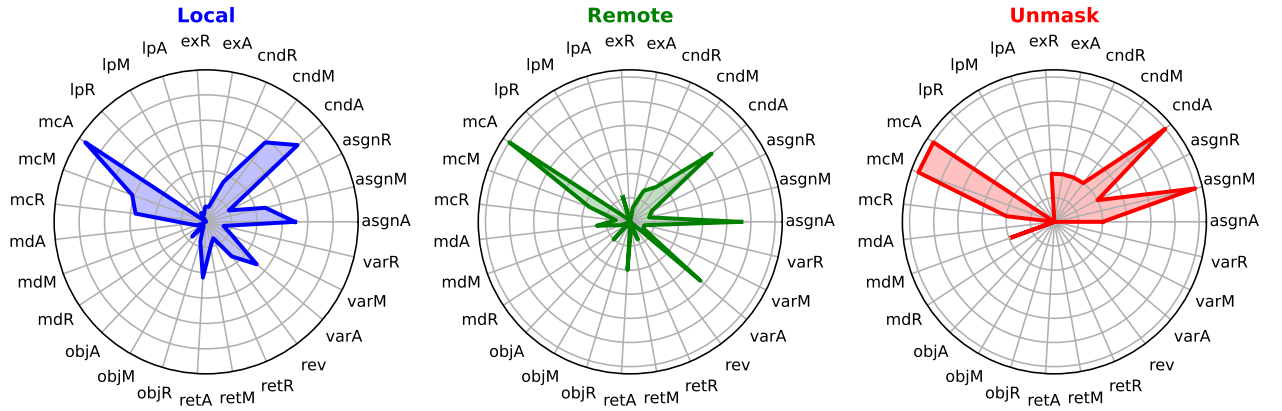


Fig. 19. Distribution of repair operators on different types of regression bug. Note that, we abbreviated the names of the repair action groups following Table 3. For example, “mcA” denotes *Method Call Addition*.

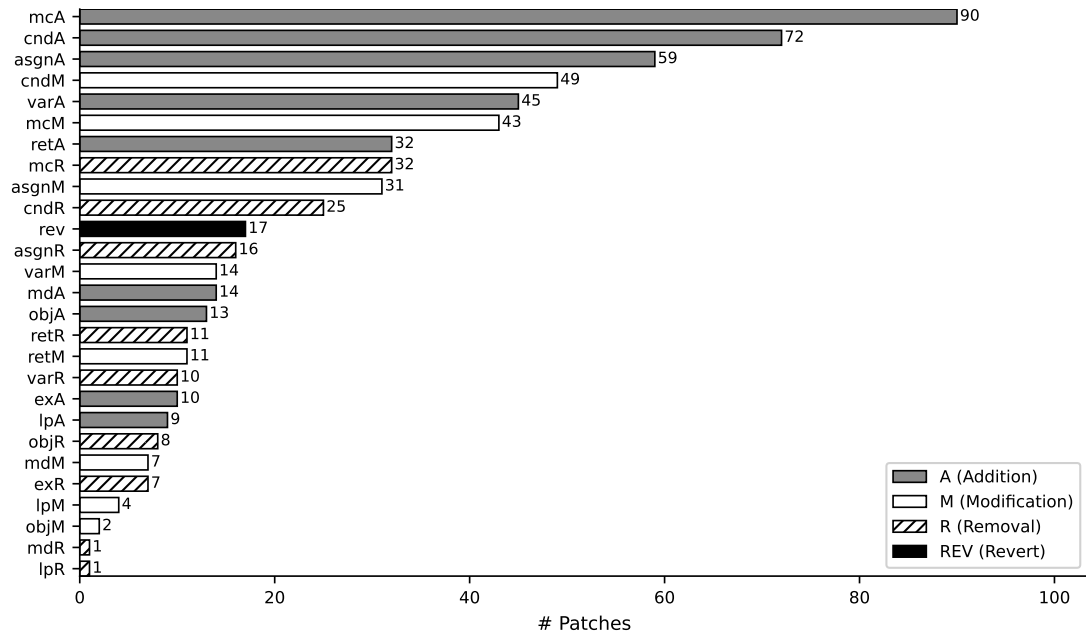


Fig. 20. Occurrence of repair operators in REGRESSIONBUG4APR. Note that, we abbreviated the names of the repair action groups following Table 3. For example, “mcA” denotes *Method Call Addition*.

actions, and black bars denote revert actions. *Method Call Addition* is the most prevalent repair operator, appearing in 90 patches, followed by *Conditional Addition* with 72 patches and *Assignment Addition* with 59 patches. The *Revert to Previous Statement* operator is exclusive to *local* regression bugs, appearing in 17 patches. In all cases, addition (54.1%) exceeds removal and modification by 36.4% and 28.5%, respectively.

We further analyze the patches that utilize the *revert to previous statement* operator. In our dataset, there are 17 regression bugs that were patched by humans using only this repair operator. Additionally, we analyze these patches to

identify the repair operators that humans would have employed if the revert operator had not been applied. Figure 21 illustrates all the repair operators used by humans to address these bugs. In total, 114 repair operators were employed to fix these 17 bugs, accounting for 17.81% of the total operators used across all bugs. This underscores the importance of recognizing this pattern in addressing regression bugs effectively.

Figure 22 presents the distribution of the number of repair operators per patch. The whiskers and outliers indicate that the data spans from 1 (lower whisker) to 9 (upper whisker), with several outliers reaching up to 15. The median (highlighted in orange) is 3, while the box extends from 2 to 5, indicating that the middle 50% of the data lies within this range. The distribution is skewed to the right, with a few patches exhibiting a much higher number of operators than the majority.

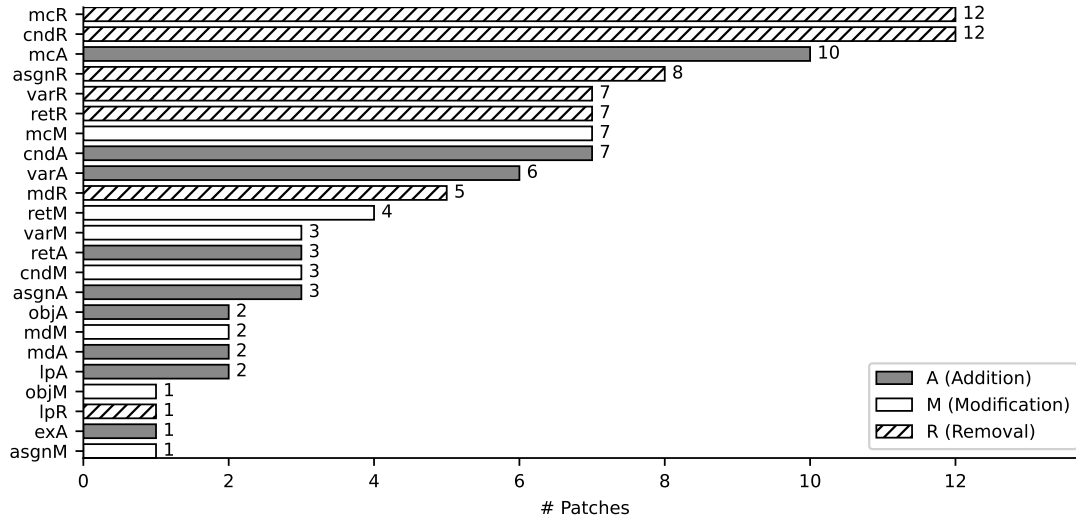


Fig. 21. Detailed examination of a repair operator, namely *revert to previous statement*. Note that, we abbreviated the names of the repair action groups following Table 3. For example, “mcA” denotes *Method Call Addition*.

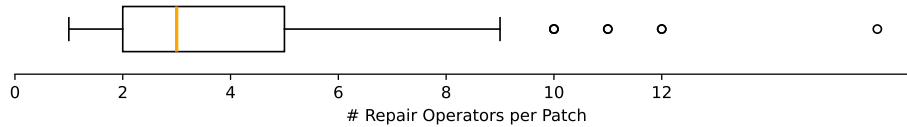


Fig. 22. Distribution of the number of repair operators per patch.

Overall, the results highlight the diversity of repair operators required to fix bugs in our dataset. Compared with Defects4J [63], our benchmark covers all repair operators observed in Defects4J. Notably, for *Local* regression bugs, the *Revert to Previous Statement* operator proves effective. Furthermore, a substantial portion of the patches involve the application of multiple repair operators, indicating the non-trivial nature of many fixes.

These results demonstrate that REGRESSIONBUG4APR captures a wide range of patch sizes and scopes. With several patches exhibiting distinctive regression-fixing patterns, the dataset includes both representative and challenging edge cases that are rarely addressed by existing tools. This diversity makes REGRESSIONBUG4APR a valuable benchmark for evaluating and advancing automated program repair techniques.

RQ1 Quality and Diversity of the RegressionBug4APR Benchmark

The analysis demonstrates the high quality of the REGRESSIONBUG4APR benchmark for APR research, highlighting key characteristics such as reality (sourced from real-world Java and Python repositories), timeliness (extending the timeframe of the most recent and relevant benchmarks), and diversity (in terms of patch size, scope, and repair operators). In addition, our structured data storage process ensures durable reproducibility of all bugs.

5 RQ2: Effectiveness of APR Techniques on Regression Bugs

Table 4. Performance Evaluation of APR Methods on the Java Regression Bugs of the REGRESSIONBUG4APR benchmark.

Method	#Plausible Patches (Plausible Rate %)	#Correct Patches (Correct Rate %)	Precision (%)
ChatGPT-4o + Conversation	60 (40.00%)	18 (12.00%)	30.00%
ChatGPT-4o + Zero-shot Prompting	35 (23.33%)	13 (8.67%)	37.14%
ChatGPT-Turbo-3.5 + Conversation	41 (27.33%)	9 (6.00%)	21.95%
ChatGPT-Turbo-3.5 + Zero-shot Prompting	28 (18.67%)	8 (5.33%)	28.57%
RepairLLaMA	31 (20.67%)	15 (10.00%)	48.39%
Incoder-6B	14 (9.33%)	5 (3.33%)	35.71%
Incoder-1B	13 (8.67%)	5 (3.33%)	38.46%
CodeGen-6B	12 (8.00%)	3 (2.00%)	25.00%
CodeGen-2B	11 (7.33%)	4 (2.67%)	36.36%
TBar	0 (0%)	0 (0%)	N/A
jGenProg	0 (0%)	0 (0%)	N/A
Cardumen	0 (0%)	0 (0%)	N/A
jKali	0 (0%)	0 (0%)	N/A
Arja	0 (0%)	0 (0%)	N/A
jMutRepair	0 (0%)	0 (0%)	N/A

To investigate the effectiveness of existing APR techniques in the context of regression errors, we evaluate both traditional and LLM-based approaches in repairing buggy programs, without providing additional regression-specific context. This setup aligns with its conventional workflow, which relies solely on the buggy program and its associated test suite for validation. Note that this research question focuses exclusively on Java bugs, as the traditional APR tools and fine-tuning-based LLM approaches evaluated here are inherently designed for Java and do not support Python. The traditional APR tools include jGenProg [36], jKali [55], Cardumen [46], Arja [82], jMutRepair [45] and TBar [39]. The LLM-based approaches include Incoder-1B and Incoder-6B [20], CodeGen-2B and CodeGen-6B [50], and RepairLLaMA [60].

Additionally, we evaluate prompt-based techniques, including zero-shot prompting and conversational APR, using two advanced models: ChatGPT-3.5-Turbo and ChatGPT-4o. The selection, configuration, and implementation details of these tools are described in Section 3.3. Table 4 summarizes the overall performance of APR techniques on the REGRESSIONBUG4APR benchmark, reporting the number of plausible patches, the number of correct patches among them, and the corresponding precision scores.

5.1 Experimental Results

Our experiments indicate that *traditional APR techniques* were unable to repair any bugs within our dataset. A deeper analysis suggests the limitation stems from two main factors: (i) the unavailability of fixes within the current version of the program, as many fixes may only exist in earlier versions [10], and (ii) the inadequacy of existing bug-fixing patterns, which do not encompass many of the bugs present in our dataset. For instance, Figures (23a) and (23b) present the bug-inducing and bug-fixing changes for *RegressionBug-84* from the REGRESSIONBUG4APR benchmark. In this case, the error emerged when developers sought to enhance system performance but inadvertently introduced implementation errors, resulting in subtle semantic bugs. The most straightforward resolution involved reverting the previous changes to eliminate the fault. However, if this bug is examined in isolation, without considering the associated regression-inducing commit, as is often the case with many APR tools, the bug-fixing pattern appears more complex, requiring numerous modifications that fall outside the scope of existing fix templates. This complexity likely explains the limited applicability of such patterns in state-of-the-art APR techniques. Conversely, if the regression-inducing commit is included in the input, a simple “*Revert to Previous Statement*” pattern could effectively resolve the issue.

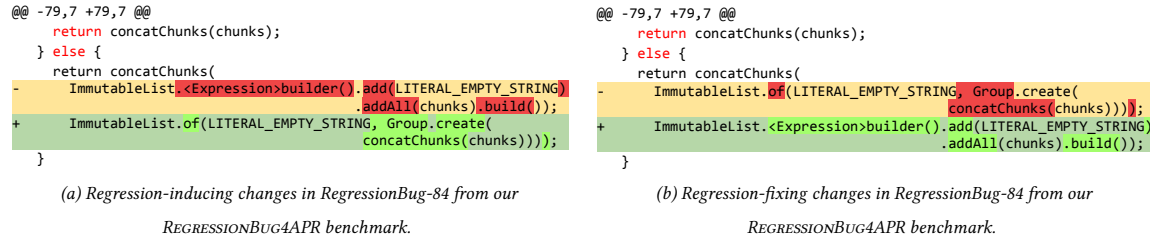


Fig. 23. Illustration of the *Revert to Previous Statement* repair operator in *RegressionBug-84* from the REGRESSIONBUG4APR benchmark.

Moving to more advanced approaches, APR techniques based on large language models demonstrate significantly improved performance over traditional tools. We categorize these techniques into two main families: *fine-tuning-based* and *prompt-based* methods. Our results show promising outcomes, some bugs can be fixed with minimal context, requiring only the buggy function and debugging information. However, other bugs necessitate richer contextual information to understand the cause of the regression and to reuse relevant variables or logic from earlier versions to construct the correct fix (e.g., reverting a previous change as in *RegressionBug-84*).

Fine-tuning-based APR tools, which employ LLMs further fine-tuned on large-scale bug-fix corpora, generate between 11 and 31 plausible patches, with 3 to 15 correct patches and precision rates ranging from 25.00% to 48.39%. Among them, RepairLLaMA achieves the highest performance, producing 15 correct patches and attaining the highest precision rate of 48.39%. Incoder outperforms CodeGen in terms of correct patches, with both Incoder-6B and Incoder-1B generating 5 correct patches each; however, Incoder-1B demonstrates slightly better precision (38.46% vs. 35.71%). For CodeGen, CodeGen-2B achieves 4 correct patches compared to 3 for CodeGen-6B, with CodeGen-2B also demonstrating higher

precision (36.36% vs. 25.00%). Notably, RepairLLaMA is able to fix every bug resolved by InCoder and CodeGen, with the exception of *RegressionBug-48* and *RegressionBug-139*, for which it generates a plausible but incorrect patch.

On the other hand, *prompt-based APR techniques*, including both *zero-shot prompting* and *conversational* strategies, also demonstrate strong performance. These methods yield between 28 and 60 plausible patches, with 8 to 18 correct patches, and precision rates ranging from 21.95% to 37.14%. Among them, ChatGPT-4o with conversational interaction achieves the highest number of correct patches (18 in total). Overall, ChatGPT-4o outperforms ChatGPT-Turbo-3.5 in terms of correct patches: ChatGPT-4o achieves 18 and 13 correct patches under the conversational and zero-shot configurations, respectively, compared to 9 and 8 for ChatGPT-Turbo-3.5. For both models, the conversational configuration generally generates more plausible patches than zero-shot prompting. Interestingly, the zero-shot prompting configuration results in higher precision for both models (37.14% vs. 30.00% for ChatGPT-4o, and 28.57% vs. 21.95% for ChatGPT-Turbo-3.5), compared to their conversational counterparts. This discrepancy may be due to the tendency of conversational strategies to overfit to the given test cases, as they rely on iterative feedback from test executions to guide patch generation, resulting in a higher number of plausible patches but relatively few correct ones among them.

SUMMARY

In sum, ChatGPT-4o (Conversation) achieves the highest number of correct repairs, successfully fixing 18 bugs, while RepairLLaMA achieves 15 correct repairs with the highest precision of 48.39%, making it the most reliable tool among all evaluated techniques. The union of both tools results in 27 unique bugs being fixed in the Java portion of the RegressionBug4APR benchmark. Notably, RepairLLaMA is able to repair 9 bugs that ChatGPT-4o (Conversation) fails to fix, while ChatGPT-4o (Conversation) is able to repair 12 bugs that RepairLLaMA fails to fix, highlighting the complementary nature of the two approaches.

5.2 Performance Breakdown by Regression Bug Type

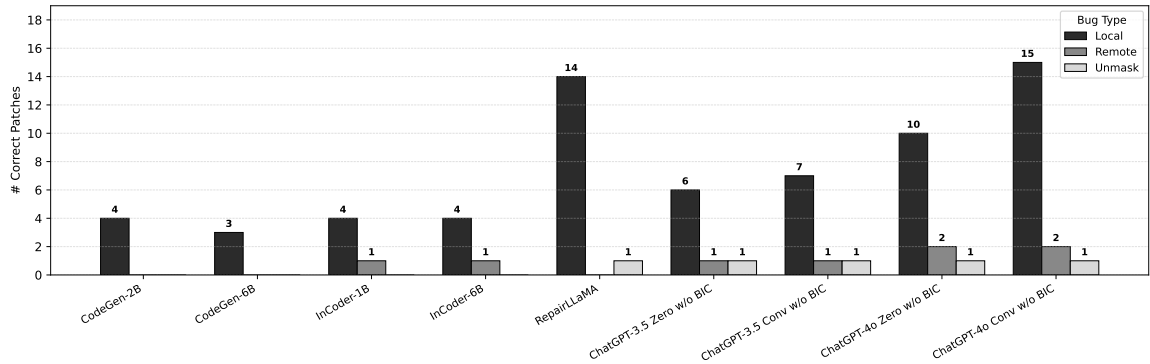


Fig. 24. Number of correct patches per regression bug type (Local, Remote, and Unmask) for each evaluated APR technique in RQ2 on the Java regression bugs of the REGRESSIONBUG4APR benchmark.

To further understand which regression bug types are most challenging for existing APR techniques, we provide a breakdown of repair results across Local, Remote, and Unmask categories on the Java of the REGRESSIONBUG4APR benchmark, as illustrated in Figures 24 and 25. The Java portion of the REGRESSION4APR benchmark comprises 112 Local, 32 Remote, and 3 Unmask Java regression bugs, with 3 additional bugs left uncategorized. No evaluated technique

successfully repairs the uncategorized bugs, as they require modifications across multiple files. Across all evaluated techniques, Local bugs consistently account for the majority of correct repairs, as shown in Figure 24. ChatGPT-4o (Conversation) achieves the highest number of Local bug repairs (15), followed by RepairLLaMA with 14. Remote bugs prove particularly challenging, with most techniques producing at most 1–2 correct repairs for this category. Unmask bugs show moderate repairability despite their small sample size, with RepairLLaMA, ChatGPT-Turbo-3.5, and ChatGPT-4o each correctly repairing 1 Unmask bug. As shown in Figure 25, Remote bugs achieve the lowest fix rate (9.4%) among all three bug types, while Local bugs achieve 22.3% and Unmask bugs achieve 33.3%.

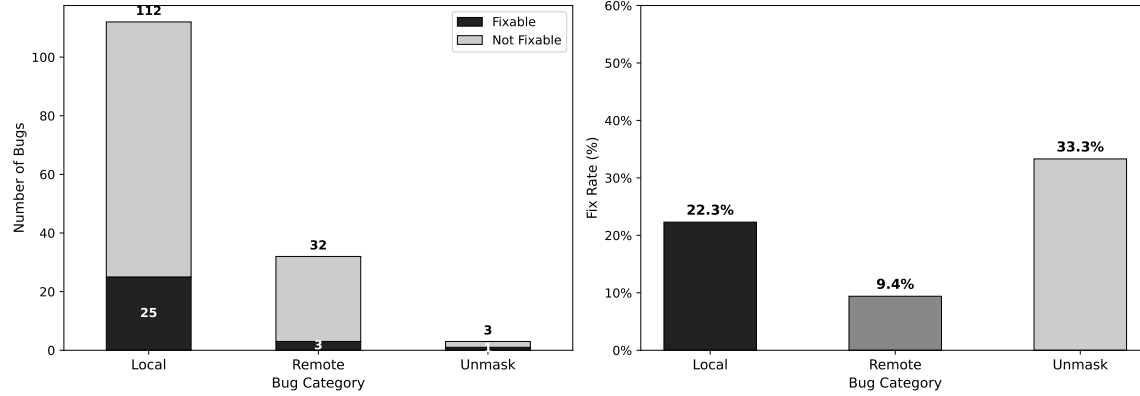


Fig. 25. Number of fixable bugs (left) and fix rate (right) per regression bug type (Local, Remote, and Unmask) across all evaluated APR techniques in RQ2 on the Java regression bugs of the REGRESSIONBUG4APR benchmark.

SUMMARY

On the Java regression bugs of the REGRESSIONBUG4APR benchmark, considering the union of all evaluated approaches without BIC context, Local bugs achieve a fix rate of 22.3% (25 out of 112), Remote bugs achieve 9.4% (3 out of 32), and Unmask bugs achieve 33.3% (1 out of 3). Remote bugs achieve the lowest fix rate among all three bug types.

5.3 Analysis of Representative Repair Instances

Bugs fixed by RepairLLaMA but not ChatGPT-4o (Conversation). RepairLLaMA is able to repair bugs that ChatGPT-4o (Conversation) fails to fix, as illustrated by *RegressionBug-95* and *RegressionBug-30*. In *RegressionBug-95*, the correct fix involves adjusting a numeric constant: `"IOUtils.getChars(nanos, 30 - (9 - nanoSize), buf);"` to `"IOUtils.getChars(nanos, 29 - (9 - nanoSize), buf);"`. This change precisely aligns the buffer offset with the expected location of the *nanos* value in timestamps formatting. Although ChatGPT-4o (Conversation) effectively utilizes the failure message to exactly pinpoint the faulty location, it generates an overfitting patch: `"IOUtils.getChars(nanos, buf.length, buf);"`. This version only passes the test case when *"nanoSize"* is 9, making it a coincidental success rather than a semantically correct fix. In *RegressionBug-30*, ChatGPT-4o (Conversation) produces a plausible patch after five attempts and three rounds of interaction. The final patch is unnecessarily complex and lacks the core corrective logic expected in the human-written fix. As a result, it introduces semantic divergence from the intended behavior, despite passing the test suite.

Bugs fixed by ChatGPT-4o (Conversation) but not RepairLLaMA. ChatGPT-4o (Conversation) is able to repair bugs that RepairLLaMA fails to fix, as illustrated by *RegressionBug-85*. ChatGPT-4o successfully repaired this bug after four attempts and two rounds of interaction. In the first round, it produced a partial fix and responded with the following insight:

"... The error message indicates that the expected output should have the 'metadata' argument only in the 'if' condition and not in the 'else' condition. The current implementation mistakenly includes 'metadata' in both branches of the conditional expression..."

Although this patch did not yet pass the test case, the response demonstrated its ability to reason about the structure of the bug. In the second round, ChatGPT-4o refined its fix with the following response:

"... To address the error you mentioned, it seems the issue lies in how the conditional expression is being constructed, particularly regarding the parameters passed to the 'construct' method for the 'SOY_VISUAL_ELEMENT'. The test case indicates that when 'GOOG_DEBUG' is 'true', the 'SOY_VISUAL_ELEMENT' should be constructed with both the ID and the name, while when 'GOOG_DEBUG' is 'false', it should be constructed with only the ID..."

This revised fix passed the test case and demonstrates how ChatGPT-4o with conversation effectively leverages failing test case information and interaction history to guide its repair process. The example highlights the benefit of interactive, test-driven repair in addressing complex semantic errors.

A difficult instance fixed by no technique. We present *RegressionBug-76* as a representative difficult instance that no evaluated technique could fix. This bug is from the *cron-utils* project, which supports Quartz cron expressions including "overflow ranges", i.e., ranges where the start value exceeds the end value, representing a wrap-around (e.g., 20–10 for seconds means seconds 20 to 59 and 0 to 10). A bug-inducing commit ("Issue #305: Fix bug regarding bad year handling for next execution") added `.withStrictRange()` to the seconds, minutes, hours, day-of-month, and year field definitions in the Quartz definition builder, inadvertently rejecting overflow ranges as invalid and breaking backward compatibility. The failures are observed below:

Prompt-based approaches consistently conflate the overflow range rejection with an off-by-one error in the valid range bounds, leading them to reduce (1, 32) to (1, 31) and (1, 13) to (1, 12). These transformations appear semantically natural from general domain knowledge but are factually incorrect in the Quartz specification, where these bounds are intentional to permit step values equal to the full field span. The few patches that correctly removed `.withStrictRange()` always compounded the error by simultaneously reducing these bounds, introducing new failures in period-step validation.

Fine-tuning-based approaches reveal a complementary failure mode. CodeGen and InCoder models produce severely truncated or structurally malformed patches, with over 80% of outputs timing out due to incomplete method bodies, and others fail to compile due to field duplication, wrong field ordering, or hallucinated API signatures (e.g., `.withStrictRange(1, 32)`). RepairLLaMA, while generating syntactically complete and compilable code (9/10 patches compile successfully), fails to produce any semantically meaningful change: all nine compilable patches unconditionally retain `.withStrictRange()` on seconds, minutes, and hours, with the only modifications being irrelevant reordering of method chains or incorrect additions of `.optional()` to mandatory fields.

The root cause is that no model produced the correct fix required by the ground truth, selectively removing `.withStrictRange()` from exactly four fields while preserving it on the year field. This distinction is derivable only from the BIC context, which is unavailable in the current repair setting. This case illustrates a fundamental limitation shared by both prompt-based and fine-tuning-based APR: when the observable test failure does not directly reflect the underlying root cause of the regression, and when the correct repair requires both library-specific semantic knowledge

and an understanding of the intent behind the bug-inducing context, existing techniques, whether based on fine-tuning or conversational prompting, are insufficient to generate the correct patch.

This example motivates future work: (i) developing repair approaches that incorporate bug-inducing context beyond the currently available information (e.g., buggy method and test failure); and (ii) incorporating domain-specific knowledge to support cases where the correct fix requires library-level understanding that cannot be inferred from the available context or the trained knowledge of LLMs.

RQ2 Effectiveness of APR Techniques on Regression Bugs

Our experimental results on 15 APR methods on the 150 Java regression bugs demonstrate that traditional techniques fail to repair any bugs in the dataset, largely due to the limitations of their underlying bug-fixing patterns. In contrast, advanced APR tools leveraging large language models show promising results. Notably, ChatGPT-4o (Conversation) achieves the highest number of correct repairs, successfully fixing 18 bugs, while RepairLLaMA attains the highest precision of 48.39%, making it the most reliable tool in terms of generating correct patches.

6 RQ3: Impact of Bug-Inducing Change Information on APR Performance

To explore whether providing additional context, specifically bug-inducing change information, can enhance regression error repair performance, we focus on prompt-based techniques and adapt the prompt design to explicitly incorporate regression-specific context. We choose this approach based on its demonstrated effectiveness (as shown in RQ2), its inherent flexibility in both model usage and prompt construction, and its greater data efficiency compared to fine-tuning-based methods. The selection, configuration, and implementation details of these techniques are provided in Section 3.3.

6.1 Experimental Results

Our experimental results show that for both base models, including ChatGPT-4o and ChatGPT-Turbo-3.5, incorporating regression-specific context, specifically bug-inducing change (BIC) information, improves the plausible rate, correct rate and precision compared to their counterparts configured without this information. Additionally, when BIC context is included, the conversational configuration consistently outperforms the zero-shot prompting configuration in both base models.

Table 5 summarizes the overall performance of prompt-based APR techniques on the Java benchmark, reporting the number of plausible patches, the number of correct patches and the corresponding precision scores. For ChatGPT-4o, the conversational strategy with BIC successfully repairs 29 bugs (19.33%), compared to only 18 bugs (12.00%) without BIC, an increase of approximately 7.33%. The precision also increases by 10.28% (from 30.00% to 40.28%). Similarly, zero-shot prompting with BIC achieves 21 correct repairs, outperforming the version without BIC, which fixes 13 bugs, with a precision gain of 4.04% (from 37.14% to 41.18%). A similar trend is observed for ChatGPT-Turbo-3.5. The conversational strategy with BIC fixes 14 bugs (9.33%), compared to 9 bugs (6.00%) without BIC, accompanied by a 4.47% increase in precision (from 21.95% to 26.42%). Likewise, zero-shot prompting with BIC results in 12 correct repairs, outperforming its counterpart without BIC, which fixes 8 bugs, with a corresponding precision gain of 6.72% (from 28.57% to 35.29%). When comparing configurations within each base model, zero-shot prompting with BIC consistently outperforms both the conversational and zero-shot configurations without BIC, further underscoring the value of incorporating regression-specific context. Lastly, it is important to note that ChatGPT-4o under conversation configuration augmented

with BIC achieves the best overall performance across all evaluated configurations, in terms of both correct repairs and precision.

Table 5. Performance Evaluation of Prompt-based APR Approaches With and Without Full Bug-Inducing Change (BIC) Information on Java regression bugs of the REGRESSIONBUG4APR benchmark.

Base Model	Method	#Plausible Patches (Plausible Rate %)	#Correct Patches (Correct Rate %)	Precision (%)
ChatGPT-4o	<i>Zero-shot Prompting w/o BIC</i>	35 (23.33%)	13 (8.67%)	37.14%
	<i>Conversation w/o BIC</i>	60 (40.00%)	18 (12.00%)	30.00%
	<i>Zero-shot Prompting with BIC</i>	51 (34.00%)	21 (14.00%)	41.18%
	<i>Conversation with BIC</i>	72 (48.00%)	29 (19.33%)	40.28%
ChatGPT-Turbo-3.5	<i>Zero-shot Prompting w/o BIC</i>	28 (18.67%)	8 (5.33%)	28.57%
	<i>Conversation w/o BIC</i>	41 (27.33%)	9 (6.00%)	21.95%
	<i>Zero-shot Prompting with BIC</i>	34 (22.67%)	12 (8.00%)	35.29%
	<i>Conversation with BIC</i>	53 (35.33%)	14 (9.33%)	26.42%

Table 6. Performance Evaluation of Prompt-based APR Approaches With and Without Full Bug-Inducing Change (BIC) Information on Python Regression Bugs of the REGRESSIONBUG4APR benchmark.

Base Model	Method	#Plausible Patches (Plausible Rate %)	#Correct Patches (Correct Rate %)	Precision (%)
ChatGPT-4o	<i>Zero-shot Prompting w/o BIC</i>	25 (50.00%)	6 (12.00%)	24.00%
	<i>Conversation w/o BIC</i>	29 (58.00%)	6 (12.00%)	20.69%
	<i>Zero-shot Prompting with BIC</i>	25 (50.00%)	7 (14.00%)	28.00%
	<i>Conversation with BIC</i>	35 (70.00%)	10 (20.00%)	28.57%
ChatGPT-Turbo-3.5	<i>Zero-shot Prompting w/o BIC</i>	9 (18.00%)	2 (4.00%)	22.22%
	<i>Conversation w/o BIC</i>	27 (54.00%)	5 (10.00%)	18.52%
	<i>Zero-shot Prompting with BIC</i>	16 (32.00%)	3 (6.00%)	18.75%
	<i>Conversation with BIC</i>	27 (54.00%)	6 (12.00%)	22.22%

Table 6 summarizes the overall performance of prompt-based APR techniques on the Python benchmark. The results are consistent with the trends observed in the Java benchmark, supporting the generalizability of our findings across programming languages. For ChatGPT-4o, incorporating BIC information consistently improves repair effectiveness: the conversational configuration with BIC achieves 10 correct patches (28.57% precision) compared to 6 (20.69% precision) without BIC, and the zero-shot configuration with BIC achieves 7 correct patches (28.00% precision) compared to 6 (24.00%) without BIC. For ChatGPT-Turbo-3.5, the conversational configuration with BIC achieves 6 correct patches

(22.22% precision) compared to 5 (18.52% precision) without BIC, while the zero-shot configuration with BIC achieves 3 correct patches (18.75% precision) compared to 2 (22.22% precision) without BIC. The limited improvement for ChatGPT-Turbo-3.5 is consistent with observations in the Java benchmark, where the benefit of BIC information is more pronounced for the stronger model. Overall, the conversational configuration with full BIC tends to achieve stronger performance, particularly for ChatGPT-4o, which is consistent with the observations in the Java benchmark. These results suggest that incorporating regression-specific context benefits repair effectiveness across programming languages, though the extent of improvement depends on the underlying model capability.

SUMMARY

In total, considering both Java and Python benchmarks, the conversational configuration of ChatGPT-4o augmented with full BIC information achieves the best overall performance, correctly repairing 39 out of 200 bugs (19.50%), compared to 24 out of 200 bugs (12.00%) without BIC, representing a 1.6 $\times$ improvement. These results demonstrate that incorporating bug-inducing change information consistently enhances repair effectiveness across both programming languages and evaluation settings.

6.2 Performance Breakdown by Regression Bug Type

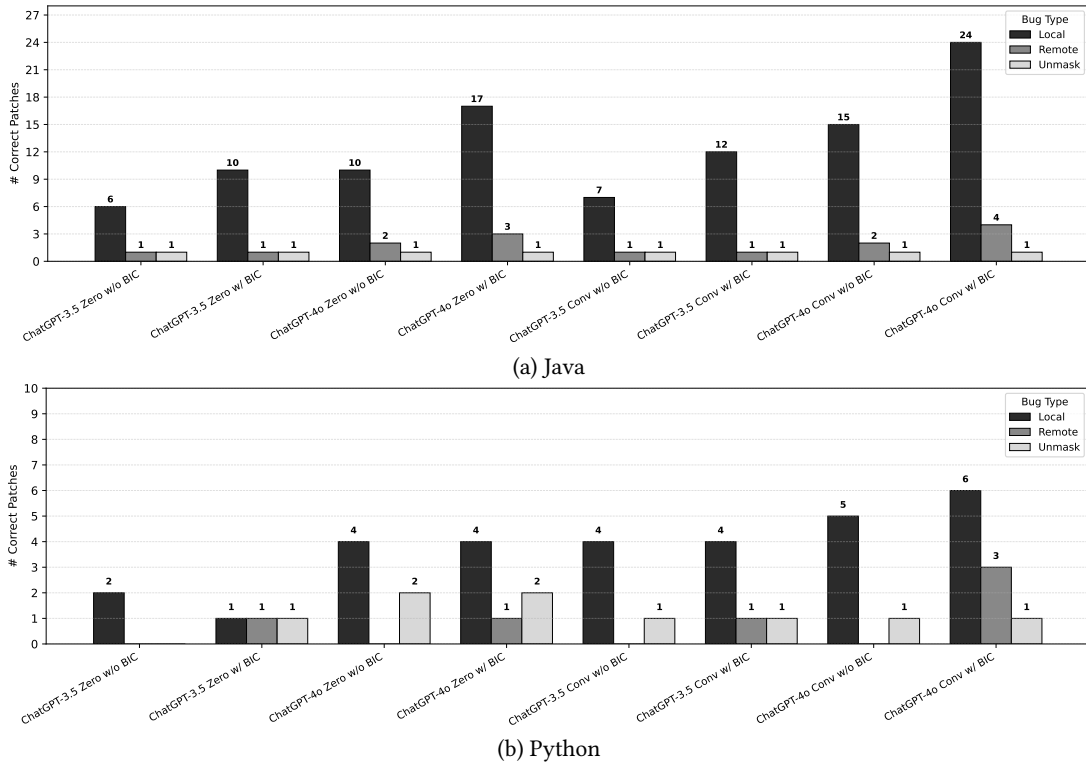


Fig. 26. Number of correct patches per regression bug type (Local, Remote, and Unmask) for each evaluated APR technique in RQ3 on the REGRESSIONBUG4APR benchmark. (a) Java. (b) Python.

To further understand how BIC information affects repair effectiveness across different regression bug types, we analyze the repair results broken down by Local, Remote, and Unmask categories on the REGRESSIONBUG4APR benchmark.

As shown in Figure 26, Local bugs consistently account for the majority of correct patches across all evaluated techniques and configurations in both languages. On the Java benchmark (Figure 26a), ChatGPT-4o (Conversation with BIC) achieves the highest overall repair count, generating 24, 4, and 1 correct patches for Local, Remote, and Unmask bugs, respectively. Notably, providing BIC context yields substantial gains across configurations; for instance, ChatGPT-4o (Conversation) increases its Local correct patches from 15 to 24 and its Remote correct patches from 2 to 4 when augmented with BIC information. On the Python benchmark (Figure 26b), ChatGPT-4o (Conversation with BIC) similarly achieves the best performance, generating 6, 3, and 1 correct patches for Local, Remote, and Unmask bugs, respectively. The benefit of BIC context is observed consistently across all bug types and technique configurations in both languages.

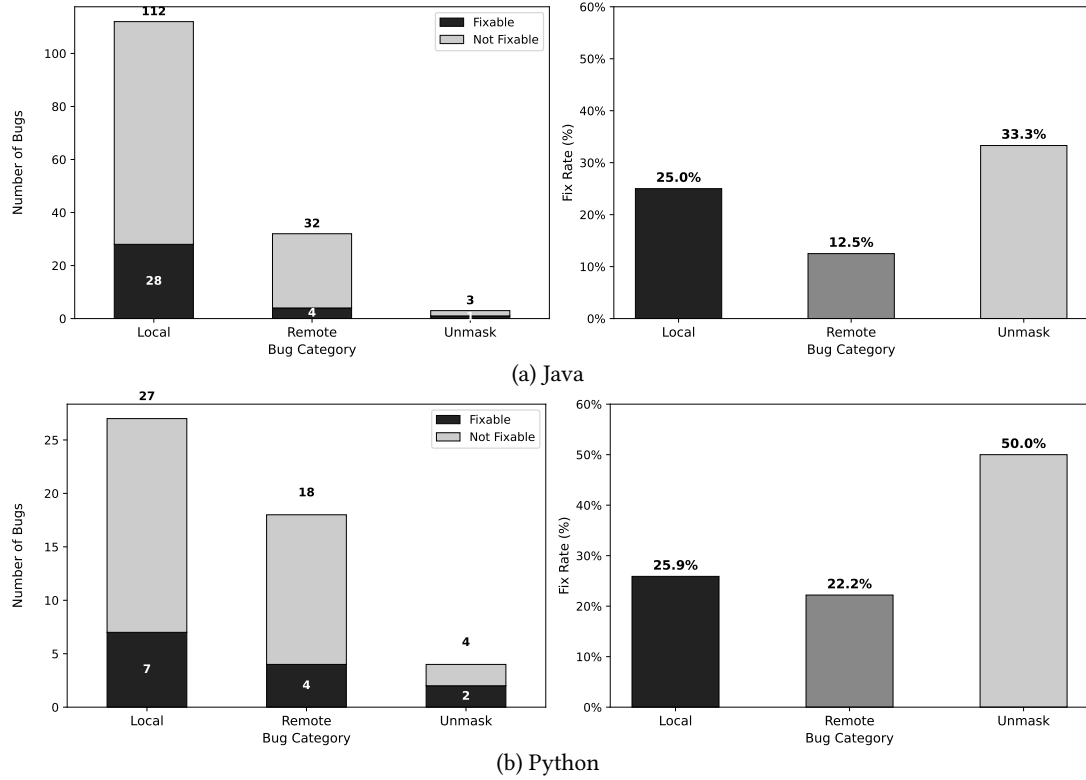


Fig. 27. Number of fixable bugs (left) and fix rate (right) per regression bug type (Local, Remote, and Unmask) across all evaluated prompt-based APR techniques incorporating BIC context on the REGRESSIONBUG4APR benchmark. (a) Java. (b) Python.

Figure 27 presents the fix rates per regression bug type for both languages. On Java (Figure 27a), a direct comparison with the RQ2 results without BIC context (Figure 25) reveals improvements across all three bug categories when BIC information is incorporated: the fix rate for Local bugs increases from 22.3% to 25.0%, the fix rate for Remote bugs increases from 9.4% to 12.5%, and the fix rate for Unmask bugs remains at 33.3% (1 out of 3 bugs in both settings). On the

Python benchmark (Figure 27b), Local bugs achieve a fix rate of 25.9% (7 out of 27), Remote bugs achieve 22.2% (4 out of 18), and Unmask bugs achieve the highest fix rate at 50.0% (2 out of 4); however, the latter result should be interpreted with caution given the limited sample size of only 4 Unmask bugs. Across both languages, Remote bugs consistently achieve the lowest fix rate among all three bug categories, suggesting that they remain the most challenging category for automated repair, even when BIC context is provided.

SUMMARY

On the Java benchmark, incorporating BIC information improves the fix rate across all regression bug types: Local bugs increase from 22.3% to 25.0%, and Remote bugs increase from 9.4% to 12.5%, while Unmask bugs remain at 33.3%. Across the REGRESSIONBUG4APR benchmark (200 bugs), Local bugs achieve an overall fix rate of 25.2% (35 out of 139 bugs), Remote bugs achieve 16.0% (8 out of 50 bugs), and Unmask bugs achieve 42.9% (3 out of 7 bugs). Remote bugs consistently represent the most challenging regression bug category for automated repair across both programming languages and all evaluated configurations.

6.3 Qualitative Analysis of Plausible Patches

To better understand how bug-inducing change (BIC) information influences the behavior of LLMs in automated program repair, we conduct a qualitative analysis of plausible patches generated by the best-performing configuration, conversational ChatGPT-4o with full BIC context. Specifically, we selected cases in which regression bugs were successfully repaired only when BIC information was included, as identified in Section 6.1. We organize our findings into representative case studies, each illustrating a distinct pattern in how the model leverages contextual and historical information to generate correct patches.

1. Understanding the Original Code:

- The original code used `ImmutableList.<Expression>builder().add(LITERAL_EMPTY_STRING).addAll(chunks).build()`.
- This approach prepends `LITERAL_EMPTY_STRING` to the list of expressions and then concatenates them.

2. Bug-Inducing Change:

- The change replaced the builder pattern with `ImmutableList.of(LITERAL_EMPTY_STRING, Group.create(concatChunks(chunks)))`.
- This modification wraps the entire concatenation of chunks within a `Group` and then concatenates it with `LITERAL_EMPTY_STRING`.

3. Issue with the Change:

- The change alters the structure of the concatenation by grouping all chunks first before concatenating with an empty string.
- This could lead to a different order of operations or unexpected coercion behaviors, especially with how expressions are evaluated and concatenated in JavaScript.

4. Root Cause:

- The grouping of all chunks before concatenating with `LITERAL_EMPTY_STRING` may be affecting the string coercion or evaluation order, leading to the failing test cases.

5. Solution:

- Revert back to the original approach that uses the builder pattern to ensure that `LITERAL_EMPTY_STRING` is directly prepended to the list of chunks, maintaining the intended concatenation order.

Fig. 28. Excerpt from ChatGPT-4o’s response under the zero-shot prompting configuration augmented with bug-inducing change information for repairing *RegressionBug-84*.

6.3.1 Observation 1: Fully reverting to previous statements guided by bug-inducing change information. In some cases, APR techniques successfully repair the bug by reverting all modified statements to the version that was functioning correctly before the regression was introduced. Generating such patches correctly requires the model to understand the broader context of both the bug-inducing changes and the buggy program. For example, the model must be able to distinguish between intended updates, such as bug fixes or feature additions, and unintended changes that should

be reverted. For instance, Figure 23 presents an illustration of the *Revert to Previous Statement* repair operator in *RegressionBug-84* from the *RegressionBug4APR* benchmark. In this case, the developer had introduced a change that wrapped a `+` expression in parentheses (e.g., `" " + (a + b))` to prevent premature string coercion. However, this modification was incorrect because the JavaScript `+` operator is not associative. The fix reverted this change by removing the parentheses and directly prepending `" "` to the list of expressions (i.e., `" " + a + b + c`), allowing JavaScript to perform left-to-right type coercion and thereby preserving the intended semantics.

Figure 28 presents an excerpt from ChatGPT-4o’s reasoning under zero-shot prompting configuration with BIC context for *RegressionBug-84*. Specifically, the model first analyzes the original code, then interprets the bug-inducing change, identifies the semantic issue it introduced, and ultimately concludes that the regression stems from replacing the builder-style concatenation with a grouped expression. This modification unintentionally alters the evaluation order and type coercion behavior in JavaScript, resulting in a failing test case. ChatGPT-4o correctly identifies the root cause and proposes reverting to the original builder-based expression to restore the intended semantics, aligning with the developer’s manual fix.

```
@@ -825,7 +835,7 @@
 * @see #copyFileToDirectory(File, File, boolean)
 * @since 2.8.0
 */
- public static void copyFile(final File srcFile, final File destFile, final boolean
preserveFileDate, final CopyOption... copyOptions) throws IOException {
+ public static void copyFile(final File srcFile, final File destFile, final boolean
preserveFileDate, CopyOption... copyOptions) throws IOException {
    Objects.requireNonNull(destFile, "destination");
    checkFileExists(srcFile, "srcFile");
    requireCanonicalPathsNotEquals(srcFile, destFile);
@@ -834,10 +844,18 @@
    checkFileExists(destFile, "destFile");
    requireCanWrite(destFile, "destFile");
}
+ final boolean isSymlink = Files.isSymbolicLink(srcFile.toPath());
+ if (isSymlink && !Arrays.asList(copyOptions).contains(LinkOption.NOFOLLOW_LINKS)) {
+     final List<CopyOption> list = new ArrayList<CopyOption>(Arrays.asList(copyOptions));
+     list.add(LinkOption.NOFOLLOW_LINKS);
+     copyOptions = list.toArray(new CopyOption[0]);
+ }
    Files.copy(srcFile.toPath(), destFile.toPath(), copyOptions);

    // On Windows, the last modified time is copied by default.
- if (preserveFileDate && !setTimes(srcFile, destFile)) {
+ if (preserveFileDate && !isSymlink && !setTimes(srcFile, destFile)) {
    throw new IOException("Cannot set the file time.");
}
}
```

(a) Bug-inducing changes introduced by a developer in *RegressionBug-99*.

```

    }
    final Path srcPath = srcFile.toPath();
    final boolean isSymlink = Files.isSymbolicLink(srcPath);
- if (isSymlink && !Arrays.asList(copyOptions).contains(LinkOption.NOFOLLOW_LINKS)) {
-     final List<CopyOption> list = new ArrayList<>(Arrays.asList(copyOptions));
-     list.add(LinkOption.NOFOLLOW_LINKS);
-     copyOptions = list.toArray(PathUtils.EMPTY_COPY_OPTIONS);
- }
+ // Instead of automatically adding NOFOLLOW_LINKS, we respect the passed copyOptions
+ // and avoid any default addition of NOFOLLOW_LINKS unless explicitly intended by the caller.
    Files.copy(srcPath, destFile.toPath(), copyOptions);
    // On Windows, the last modified time is copied by default.
    if (preserveFileDate && !Files.isSymbolicLink(srcPath) && !setTimes(srcFile, destFile)) {

```

(b) Correct patch generated by ChatGPT-4o via conversational prompting augmented with full BIC information.

Fig. 29. Illustration of the *Partially Revert to Previous Statement* repair operator applied to *RegressionBug-99*: (a) the bug-inducing changes and (b) the corresponding correct patch generated by ChatGPT-4o under the conversational configuration augmented with full bug-inducing change information.

6.3.2 Observation 2: Partially reverting edits guided by bug-inducing change information. Beyond full reversion, partially reverting selected code fragments also proves effective in cases where only specific statements introduced in the bug-inducing commit need to be undone. This strategy allows the model to preserve valid modifications while selectively removing only the changes responsible for the regression. For example, Figure 29 presents an illustration of the *Partially Revert to Previous Statement* repair operator in *RegressionBug-99*, as generated by ChatGPT-4o under the conversation configuration enhanced with bug-inducing change information. In this case, the developers changed the behavior of `copyFile` to copy symbolic links itself (i.e., the path to the target), rather than dereferencing the link and copying the actual target file. This change reversed the default behavior expected by many systems and broke existing code that relied on the original semantics. As discussed in the commit history¹⁹, the developers chose to revert only the

¹⁹ Available at: <https://github.com/apache/commons-io/commit/ec4144b01b4107d6b39f5f4d784cf05217ea4dfa>

code responsible for the bug, while retaining the condition `"if (preserveFileDate && !isSymLink && !setTimes(srcFile, destFile)) {"`. The retained code does not change behavior, it simply avoids setting file timestamps on symbolic links to avoids errors related to modifying *symlink* metadata, which is often OS-dependent and error-prone. This careful partial reversion ensures correctness without discarding unrelated, valid improvements.

The bug-inducing commit message mentions changes related to copying symbolic links. The function `copyFile` has been modified to handle symbolic links with the introduction of the `NOFOLLOW_LINKS` option, which means that it should copy the link itself rather than the file it points to. The failing test case indicates an issue with this behavior: the expected behavior was not met.

The primary issue seems to stem from the logic handling symbolic links. The function was updated to ensure that if a symbolic link is detected, the `NOFOLLOW_LINKS` option is included in the copy options. However, it seems that this logic either doesn't work as intended or causes unexpected behavior in some circumstances.

Let's review the critical part of the function concerning symbolic links:

```
```java
final boolean isSymLink = Files.isSymbolicLink(srcPath);
if (isSymLink && !Arrays.asList(copyOptions).contains(LinkOption.NOFOLLOW_LINKS)) {
 final List<CopyOption> list = new ArrayList<>(Arrays.asList(copyOptions));
 list.add(LinkOption.NOFOLLOW_LINKS);
 copyOptions = list.toArray(PathUtils.EMPTY_COPY_OPTIONS);
}
```
```

This logic is supposed to check if the source file is a symbolic link and if the `NOFOLLOW_LINKS` option is not already present in the `copyOptions`, it adds the `NOFOLLOW_LINKS` option. However, the test failure suggests that there might be a misalignment in the expected behavior when dealing with symbolic links.

To fix the issue, we need to ensure that the symbolic link behavior aligns with the test expectations. The test failure suggests that copying symbolic links should not automatically add the `NOFOLLOW_LINKS` option unless explicitly specified in the `copyOptions` parameter. Here's how we can adjust the function:

This change respects the original intent of the `copyOptions` provided by the caller and does not automatically modify them to include `NOFOLLOW_LINKS`. This should align with the expected behavior tested in the failing test case.

Fig. 30. Excerpt from ChatGPT-4o's response under the conversation configuration augmented with full bug-inducing change information for repairing *RegressionBug-99*.

Figure 30 presents an excerpt from ChatGPT-4o's reasoning under conversation configuration with BIC context for *RegressionBug-99*. Specifically, the model explains that the regression stems from logic that forcibly adds the `NOFOLLOW_LINKS` option when a symbolic link is detected, thereby overriding the caller's intended behavior. It correctly identifies that this automatic injection violates expectations, particularly in cases where callers rely on the default behavior of dereferencing symbolic links. Guided by the bug-inducing change information, the model proposes reverting the conditional block responsible for modifying the copy options, while preserving the surrounding logic, including the check that skips timestamp setting for symbolic links. This patch aligns with the human-written fix, as it reverts only the faulty logic introduced in the commit while retaining valid behavior.

6.3.3 Observation 3: Fully or Partially reverting edits could generate the plausible but incorrect patches. Apart from *Observation 1* and *2*, our patch validation process revealed recurring patterns, notably *fully or partially reverting to previously correct statements*, that lead to patches which pass the test suite but are semantically incorrect (i.e., plausible but incorrect patches). These patterns frequently appear in patches generated by techniques that leverage bug-inducing change information. While these patterns often produce plausible patches that satisfy the test suite, they may reflect an over-reliance on syntactic rollback strategies and suggest that the model lacks a deeper understanding of the contextual semantics required to correctly fix the underlying bug.

```

@@ -300,9 +310,14 @@
}

private void readUntilEndOfEntry() throws IOException {
+ if (localFileHeader.getCompressedSize() == 0) {
+     return;
+ }
+
    if (endOfEntryBuffer == null) {
        endOfEntryBuffer = new byte[512];
    }
+
    while (read(endOfEntryBuffer) != -1);
}

```

(a) Bug-inducing changes introduced by a developer
in *RegressionBug-2*.

```

@@ -303,7 +302,10 @@
}

private void readUntilEndOfEntry() throws IOException {
- if (localFileHeader.isDirectory() ||
+ if (localFileHeader.isDirectory() ||
+     localFileHeader.getCompressedSize() == 0) {
+     // If data descriptor exists, proceed to reading the
+     // entry, or else skip if the entry is directory
+     // or compressed size is 0
+     if (!localFileHeader.isDataDescriptorExists()
+         && (localFileHeader.isDirectory() ||
+             localFileHeader.getCompressedSize() == 0)) {
+         return;
+     }
+
    if (endOfEntryBuffer == null) {

```

(b) Ground-truth bug-fixing changes manually written by a developer
in *RegressionBug-2*.

Fig. 31. The bug-inducing and bug-fixing changes in *RegressionBug-2* from REGRESSIONBUG4APR benchmark.

To identify the root cause of the regression bug, let's analyze the code and the changes made:

1. Original Functionality:

- The function `readUntilEndOfEntry()` was designed to skip over data until the end of a compressed entry in a file.

2. Bug-Inducing Change:

- The additional condition `if (localFileHeader.getCompressedSize() == 0) { return; }` was introduced. This means that if the compressed size of the entry is zero, the function will return immediately without reading any further.

3. Impact of the Change:

- The change causes the function to exit early for entries that have a compressed size of zero.
- While this seems logical at first (since there's no compressed data to read), it overlooks the fact that sometimes an entry might need to be processed in a specific way even if its compressed size is zero, especially when dealing with certain ZIP file structures or when metadata needs to be processed.

4. Failing Test Case:

- The test case `testReadingZipBySkippingDataCreatedWithJDKZipReadsAllEntries` expects to read all entries correctly, including those with a compressed size of zero. The error shows that only 1 entry was read instead of the expected 6, which indicates that the function is prematurely returning and not processing all entries correctly.

5. Solution:

- We need to ensure that entries with a compressed size of zero are still processed correctly. This means removing or modifying the condition that causes the function to return early.

Here's the corrected function without the problematic condition:

```

'''java
...
'''

```

By removing the condition `if (localFileHeader.getCompressedSize() == 0) { return; }`, the function will now correctly attempt to read through the end of each entry, even if the compressed size is zero. This should resolve the regression issue and allow all entries to be processed as expected.

Fig. 32. Excerpt from ChatGPT-4o's response under the conversation configuration augmented with full bug-inducing change information for repairing *RegressionBug-2*.

To illustrate this behavior, we examine *RegressionBug-2* alongside the response from ChatGPT-4o under zero-shot prompting configuration augmented with BIC context, as shown in Figures 31 and 32. In this case, the regression-inducing changes were introduced to prevent read errors when handling JAR files using ZIP4J²⁰, but they unintentionally triggered a new bug. The issue occurred when only the first entry in a ZIP archive could be read, and subsequent entries were skipped. The regression was caused by logic that returned early if an entry's compressed size was zero (Figure 31a).

²⁰ Available at: <https://github.com/srikanth-lingala/zip4j/issues/194>

However, according to the ZIP specification, this is valid behavior when the third bit of the general purpose flag is set. This bit indicates that the actual compressed size is not available in the local file header but will instead be provided later in a separate data descriptor. To fix this, the condition was refined to be more precise: entries are now skipped only if they do not use a data descriptor and are either directories or have a compressed size of zero (Figure 31b). This update preserves the original safeguard against invalid entries while allowing valid entries that rely on the data descriptor mechanism to be processed correctly.

The ChatGPT model correctly identified the root cause of the bug: the change caused valid ZIP entries to be skipped when their compressed size was zero, resulting in only one out of six entries being read. This occurred because the code returned early without properly handling ZIP structures that rely on data descriptors or contain essential metadata (as seen in parts 3 and 4 of the excerpt in Figure 32). However, the model proposed removing the condition entirely, which would allow entries to be read even if the compressed size is zero. While this may avoid the regression error, it overlooks the original intent of the change.

```
@@ -1284,12 +1284,59 @@
    }
    final FieldInfo fieldInfo = fieldDeser.fieldInfo;
+   Field field = fieldDeser.fieldInfo.field;
    Type paramType = fieldInfo.fieldType;

+   if (paramType == boolean.class) {
+       if (value == Boolean.FALSE) {
+           field.setBoolean(object, false);
+           continue;
+       }
+       if (value == Boolean.TRUE) {
+           field.setBoolean(object, true);
+           continue;
+       }
+   } else if (paramType == int.class) {
+       if (value instanceof Number) {
+           field.setInt(object, ((Number) value).intValue());
+           continue;
+       }
+   } else if (paramType == long.class) {
+       if (value instanceof Number) {
+           field.setLong(object, ((Number) value).longValue());
+           continue;
+       }
+   } else if (paramType == float.class) {
+       if (value instanceof Number) {
+           field.setFloat(object, ((Number) value).floatValue());
+           continue;
+       }
+   } else if (paramType == double.class) {
+       if (value instanceof Number) {
+           field.setDouble(object, ((Number) value).doubleValue());
+           continue;
+       } else if (value instanceof String) {
+           double doubleValue = Double.parseDouble((String) value);
+           field.setDouble(object, doubleValue);
+           continue;
+       }
+   } else if (value != null && paramType == value.getClass()) {
+       field.set(object, value);
+       continue;
+   }
    String format = fieldInfo.format;
    if (format != null && paramType == java.util.Date.class) {
        value = TypeUtils.castToDate(value, format);
    } else {
-       value = TypeUtils.cast(value, paramType, config);
+       if (paramType instanceof ParameterizedType) {
+           value = TypeUtils.cast(value, (ParameterizedType) paramType, config);
+       } else {
+           value = TypeUtils.cast(value, paramType, config);
+       }
    }
    fieldDeser.setValue(object, value);
```

Fig. 33. Bug-inducing changes introduced by a developer in *RegressionBug-69* from REGRESSIONBUG4APR benchmark.

6.3.4 Observation 4: Combining test case feedback with bug-inducing change information to narrow the fix scope. In some cases, when the model correctly understands the context of bug-inducing changes, such as feature additions or prior bug fixes, it avoids fully or partially reverting to a previously working version. Instead, it analyzes the current buggy function while simultaneously reasoning about the root cause of the regression. This combined reasoning enables more accurate fault localization, as the bugs happens after the relevant change is introduced, making the bug-inducing changes a strong signal for identifying the issue. This, in turn, supports the generation of targeted and minimal patches. This behavior is observed in *RegressionBug-30* and *RegressionBug-69*, where failing test cases provide useful information the bugs, while the bug-inducing changes provide complementary context that helps the model localize the root cause. By leveraging both sources of information, the model is able to narrow the fix scope and avoid unnecessary or overly broad modifications.

To illustrate this behavior, we present *RegressionBug-69*, where the developers attempted to improve the performance of `JSONObject.toObject` by introducing optimizations²¹, as shown in Figure 33. However, the modification introduced a regression due to a mismatch between the field type and the expected parameter type. Specifically, the checks used primitive types, while the actual `paramType` values were boxed class types. As a result, the new conditions failed to trigger as intended, leading to incorrect behavior during deserialization.

ChatGPT-4o, under both configurations, with and without bug-inducing change (BIC) information, was able to identify the type mismatch as the root cause of the failure by analyzing the test case’s error message. However, the version augmented with BIC context demonstrated a deeper understanding of the problem and provided a clearer explanation, as shown in Figures 34a and 34b.

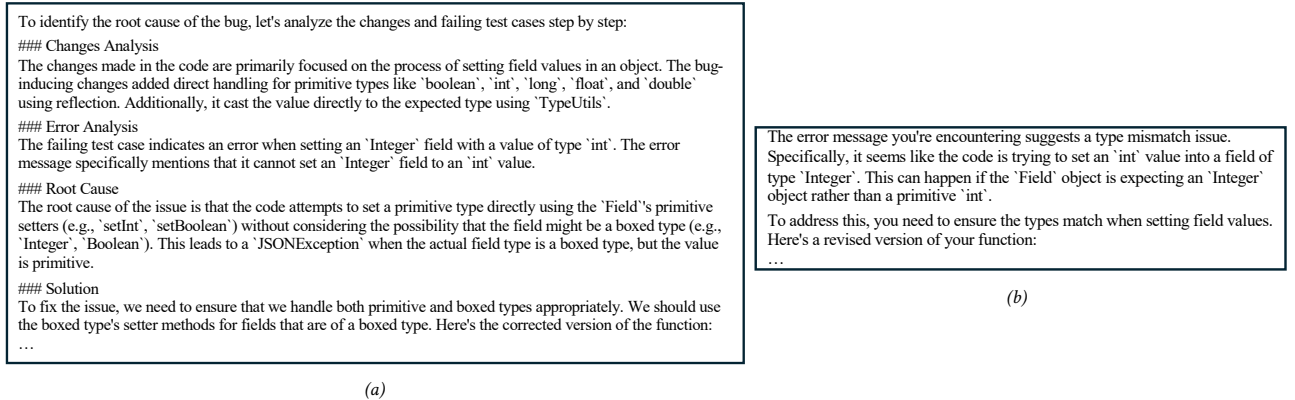


Fig. 34. Excerpts from ChatGPT-4o’s response under the conversational configuration for repairing *RegressionBug-69*: (a) with bug-inducing change information and (b) without BIC information.

More notably, when comparing the patches generated under both settings in Figures 35a and 35b, both configurations produced a correct fix at the appropriate location. Nevertheless, the patch generated without BIC context included unnecessary edits to structurally similar but semantically unrelated code. Although the resulting patch passed the test suite, the added changes introduced redundancy. This illustrates how the BIC context helps the model focus on the minimal fix necessary to resolve the regression without introducing superfluous modifications.

²¹ Available at: <https://github.com/alibaba/fastjson/commit/7b416faa1015ce04505e88d1f9b0575bcf13657f>

```

@@ -1,7 +1,5 @@
- public Object createInstance(Map<String, Object> map, ParserConfig config) //
- throws IllegalArgumentException,
- IllegalAccessException,
- InvocationTargetException {
+ public Object createInstance(Map<String, Object> map, ParserConfig config)
+ throws IllegalArgumentException, IllegalAccessException, InvocationTargetException {
    Object object = null;
    if (beanInfo.creatorConstructor == null && beanInfo.factoryMethod == null) {
        object = createInstance(null, clazz);
    }
@@ -16,28 +14,28 @@
    Field field = fieldDeser.fieldInfo.field;
    Type paramType = fieldInfo.fieldType;
    if (field != null) {
        if (paramType == boolean.class) {
+         if (paramType == boolean.class || paramType == Boolean.class) {
            if (value == Boolean.FALSE) {
                field.setBoolean(object, false);
            } else {
                field.setBoolean(object, true);
            }
        } else if (paramType == int.class) {
+         if (paramType == int.class || paramType == Integer.class) {
            if (value instanceof Number) {
                field.setInt(object, ((Number) value).intValue());
            } else {
                field.set(object, ((Number) value).intValue());
            }
        } else if (paramType == long.class) {
+         if (paramType == long.class || paramType == Long.class) {
            if (value instanceof Number) {
                field.setLong(object, ((Number) value).longValue());
            } else {
                field.set(object, ((Number) value).longValue());
            }
        } else if (paramType == float.class) {
+         if (paramType == float.class || paramType == Float.class) {
            if (value instanceof Number) {
                field.setFloat(object, ((Number) value).floatValue());
            } else {
                field.set(object, ((Number) value).floatValue());
            }
        } else if (value instanceof String) {
            String strVal = (String) value;
            floatValue = Float.parseFloat(strVal);
            field.setFloat(object, floatValue);
        } else if (paramType == double.class) {
+         if (paramType == double.class || paramType == Double.class) {
            if (value instanceof Number) {
                field.setDouble(object, ((Number) value).doubleValue());
            } else if (value instanceof String) {
                String strVal = (String) value;
                doubleValue = Double.parseDouble(strVal);
                field.setDouble(object, doubleValue);
            }
        } else if (value != null && paramType == value.getClass()) {

```

(a) Patch generated by ChatGPT-4o under the conversation configuration augmented with bug-inducing change information.

```

@@ -102,21 +100,21 @@
    Object param = map.get(fieldInfo.name);
    if (param == null) {
        Class<?> fieldClass = fieldInfo.fieldClass;
        if (fieldClass == int.class) {
+         if (fieldClass == int.class || fieldClass == Integer.class) {
            param = 0;
        } else if (fieldClass == long.class) {
+         if (fieldClass == long.class || fieldClass == Long.class) {
            param = 0L;
        } else if (fieldClass == short.class) {
+         if (fieldClass == short.class || fieldClass == Short.class) {
            param = Short.valueOf((short) 0);
        } else if (fieldClass == byte.class) {
+         if (fieldClass == byte.class || fieldClass == Byte.class) {
            param = Byte.valueOf((byte) 0);
        } else if (fieldClass == float.class) {
+         if (fieldClass == float.class || fieldClass == Float.class) {
            param = Float.valueOf(0);
        } else if (fieldClass == double.class) {
+         if (fieldClass == double.class || fieldClass == Double.class) {
            param = Double.valueOf(0);
        } else if (fieldClass == char.class) {
+         if (fieldClass == char.class || fieldClass == Character.class) {
            param = '0';
        } else if (fieldClass == boolean.class) {
+         if (fieldClass == boolean.class || fieldClass == Boolean.class) {
            param = false;
        }
        if (missFields == null) {

```

(b) Patch generated by ChatGPT-4o under the conversation configuration without bug-inducing change information. This patch includes additional edits not present in (a), while other parts remain similar to (a).

Fig. 35. Patches generated by ChatGPT-4o for *RegressionBug-69* under different configurations.

6.3.5 Others: Apart from the specific cases discussed earlier, we also observed two cases, including *RegressionBug-59* and *RegressionBug-65*, in which the bug-inducing change information contributed to the patch generation or model reasoning to varying degrees. In these cases, the benefit of incorporating regression-specific context was less evident.

Specifically, in *RegressionBug-59*, the bug-inducing change introduced an entirely new method, `convexHull()`, intended to fix a prior issue. Unfortunately, the regression occurred within the newly added code itself. The accompanying commit message, "*Stolstov/issue 172 (#174)*", provided little semantic guidance, making it difficult for the model to infer the intended behavior or reasoning behind the change. Under the conversational configuration, ChatGPT-Turbo-3.5

required 8 attempts and 3 rounds of interaction to produce a plausible patch. In the initial round, the model recognized the problem through the test failure message, which included a `java.lang.NullPointerException` and noted that the `convexHull()` method failed to handle certain geometry types correctly. Despite this, the bug-inducing change context offered limited additional support, and a valid patch was only produced after two additional rounds of interaction driven by test case feedback.

To address the issue, let's take a step-by-step approach:

- 1. Understand the Functionality:**
 - The `'lowerCase'` function is designed to convert a given string `'input'` to lowercase using the `'Locale.ENGLISH'` locale.
- 2. Examine the Bug-Inducing Commit:**
 - The commit message indicates that the `'Attributes'` were refactored from a `'LinkHashSet'` to an `'array pair'`. This suggests changes in how data is stored or accessed within the system.
- 3. Analyze the Impact:**
 - The refactoring from `'LinkHashSet'` to an array could affect how strings or attributes are handled, potentially impacting the input to the `'lowerCase'` function.
 - If the input type or the way inputs are processed changed due to this refactor, it might lead to unexpected behavior or errors.
- 4. Consider Potential Issues:**
 - The function itself has no apparent bug if the input is a valid `'String'`. However, changes elsewhere might have altered the input type or its validity.
 - Possible issues could include:
 - Null inputs due to missing elements in the array.
 - Non-string inputs being passed, if the data structure was changed.
 - Incorrect handling of empty or special-case inputs.
- 5. Fix the Root Cause:**
 - Ensure that the input to the `'lowerCase'` function is always a valid `'String'`.
 - Incorporate checks or conversions if necessary to handle the updated data structure.
- 6. Propose the Corrected Function:**
 - Add a check for null inputs to prevent `'NullPointerException'`.
 - Ensure any necessary conversions or validations are in place before using `'toLowerCase'`.

Fig. 36. Excerpt from ChatGPT-4o's response under zero-shot prompting configuration augmented with bug-inducing change information for repairing *RegressionBug-65*.

In *RegressionBug-65*, this case represents a *remote* regression bug, where code modification introduces a bug in an unchanged program element, leading to failures elsewhere in the code. The bug-inducing change information in this case includes only a vague commit message: *"Refactored Attributes to be an array pair vs LinkHashSet. Also a couple perf (cpu / garbage) tweaks"*²². Based on this context, the model inferred that the `Attributes` class had been refactored from `LinkHashSet` to an array-based structure, implying a change in how attribute data is stored or accessed (*Point 2 in Figure 36*). Despite the limited context, the model, guided by both the commit message and the test failure, hypothesized potential causes such as incorrect data types or null values handling (*Point 4 in Figure 36*). It then correctly diagnosed the issue and generated an appropriate fix. This case demonstrates that even minimal bug-inducing context, when interpreted in conjunction with test case feedback, can help the model understand the underlying cause and produce a correct patch.

SUMMARY

Qualitative analysis further reveals that BIC context helps the model more effectively identify the root cause of regressions and decide whether a full or partial reversion to prior code is appropriate. However, such reversion must be applied judiciously, guided by a proper understanding of the bug context. Additionally, BIC context helps narrow the fix scope, enhancing fault localization and reducing unnecessary modifications.

²² Available at: <https://github.com/jhy/jsoup/commit/ea1fb65e9ff8eee82c4e379dc3236d09a5ab02e1>

6.4 A Systematic Classification of LLM Repair Failures

To gain a deeper understanding of the common issues and patterns found in LLM repair failures, we selected the best-performing technique in our study, specifically conversational APR with full bug-inducing change information using ChatGPT-4o model, and conducted a qualitative analysis of failure cases using open card-sorting [65]. Open card-sorting has been widely used in prior software engineering studies [34, 41, 42] to generate categories or taxonomies from empirical data.

Study Design. We selected a representative random sample of 59 bug instances (approximately 30% of the 200 bug instances) drawn from four groups: (i) non-plausible patches, (ii) plausible but incorrect patches with 1 conversation iteration, (iii) plausible but incorrect patches with 2–3 iterations, and (iv) plausible but incorrect patches with 4–5 iterations. To maintain representativeness, we preserved the same language ratio as the extended benchmark (1 Python : 3 Java), resulting in 16 Python and 43 Java instances. We note that group (iv) contained only 7 Java instances due to the limited number of bugs in this category. Each instance was documented as a card containing the bug ID, the number of repair attempts and conversation rounds, and the full repair log, including the failure type, the LLM response, and the generated patch. We then applied open card-sorting to these cards through two phases. In the first phase (*independent sorting*), two authors independently reviewed all 59 cards and assigned a descriptive name or theme to each based on the nature of the failure observed. In the second phase (*discussion and consolidation*), the two authors compared their labellings, discussed disagreements, and converged on a final label for each card, with one author serving as a moderator. Category names were derived from patterns observed consistently across the groups.

Results. To validate the reliability of our classification, we measured inter-rater agreement using Cohen’s Kappa [15], obtaining a value of $\kappa = 0.69$, which reflects substantial agreement. All disagreements were resolved through discussion. We observed that disagreements were most frequent in groups (i) *non-plausible patches* and (iv) *plausible but incorrect patches requiring 4 - 5 iterations*, where authors needed to trace through multiple conversation rounds to identify the true root cause of failure, making classification inherently more effortful and subjective. Through this analysis, we identified four categories of LLM repair failures:

(i) *Wrong Diagnosis (44.07%)*. The LLM misidentifies the root cause of the bug, leading to patches that target the wrong code location or apply incorrect fix logic. This failure arises from several causes, including limited reasoning capability, misdirection from test case names or error signals, or incorrect application of reversion strategies guided by the bug-inducing information. For example, in *RegressionBug-24*²³, the correct fix requires replacing `this.output.close()` with `this.output.flush()`. However, the test case is named `leftInputUnclosed()`, which misdirected the LLM toward `this.input.close()`. The LLM either removed this line or added a conditional check around it. As a result, it missed the true root cause entirely.

(ii) *Weak Test Suite (23.73%)*: The patch passes all provided tests but is semantically incorrect (over-fitting), indicating the test suite is insufficient to distinguish the true fix. This can arise due to insufficient coverage of boundary conditions or alternate code paths, where the LLM exploits test gaps by simplifying logic rather than correctly addressing the underlying fault. For example, in *RegressionBug-150*²⁴, the bug spans two locations in `getMinConcordantFragmentSize()`: the stream aggregation uses `.max()` instead of `.min()`, and inside the `if (getFile().exists())` block, `Math.max(super.getMaxConcordantFragmentSize(), ...)` should be `Math.min(super.getMinConcordantFragmentSize(), ...)`. The correct fix addresses both locations with targeted one-line changes while preserving the block. However, the LLM correctly fixes the stream aggregation but removes the entire `if (getFile().exists())` block rather than correcting

²³ Available at: <https://brojackvn.github.io/RegMiner4APR-Homepage/#!/bug/RegressionBug-24/RegressionBug-24>

²⁴ Available at: <https://brojackvn.github.io/RegMiner4APR-Homepage/#!/bug/RegressionBug-150/RegressionBug-150>

the logic within it. Because the test does not set up an existing assembly file, `getFile().exists()` always returns false and the deleted block is never entered, so both the correct fix and the over-removal return the same value. The critical behavioral difference, i.e., what happens when the assembly file exists, is never exercised by the test suite, allowing the structurally incorrect patch to pass all tests and appear valid.

(iii) *Unreachable Fix (20.34%)*: The correct fix requires information that is neither visible to the LLM from the method body alone nor derivable from the error signal, such as domain-specific constants or internal method identifiers, making it structurally impossible to generate the correct patch from the available context. For example, in *RegressionBug-122*²⁵, the correct fix inserts additional static field and enum constant discovery logic into the FIELD case of `getPropertyGet()` using a framework-internal API. The LLM correctly localizes the problem to the FIELD case and understands that static fields and enum constants require special handling; however, it lacks the specific framework knowledge necessary to implement the fix. It does not know which existing class handles static field introspection, what the complete `JexlPropertyGet` interface contract requires, or that the project enforces Java 1.6 source compatibility. As a result, the LLM hallucinated non-existent classes such as `EnumGetExecutor` and `StaticFieldGetExecutor`, and its attempts to use lambda expressions were rejected by the compiler due to the version constraint, producing compilation errors across nearly all 50 repair attempts and yielding no successful patch. All the information needed to generate the correct fix exists in the codebase, but none of it is visible from the provided method body alone.

(iv) *Over-repair (11.86%)*: The LLM makes excessive modifications beyond what is needed, removing or restructuring correct code when only a minimal targeted change was required. The following example, in *RegressionBug-101*²⁶, the ground-truth fix consists of a single readable-bytes guard inserted immediately before the `decodeStatus` call inside the `MSG_STATE` branch, ensuring that at least 2 bytes remain in the buffer before the status short is read. Instead, as illustrated in Figure 37, the LLM added six guards across four separate read operations throughout the entire method. Three of these guards are unnecessary, as the `MSG_ALARM` alarm byte and the `MSG_STATE` status-type byte are always present in valid messages and do not require guarding. Worse, the LLM also wrapped the `buf.readUnsignedInt()` device-time read in a guard. This introduces a new semantic error: when 2–3 bytes remain after the status-type byte, the guard skips the device-time read without consuming those bytes, so the status read then consumes device-time bytes as status data and produces a wrong position record. The test suite still passes because no test covers this partial-data scenario.

```

    } else if (type == MSG_ALARM) {
+   if (buf.readableBytes() >= 1) { // ← unnecessary guard
        position.set(Position.KEY_ALARM, decodeAlarm(buf.readUnsignedByte()));
+   }
    }
    else if (type == MSG_STATE) {
+   if (buf.readableBytes() >= 1) { // ← unnecessary guard
        int statusType = buf.readUnsignedByte();
        position.set(Position.KEY_EVENT, statusType);
        if (statusType == 0x01 || statusType == 0x02 || statusType == 0x03) {
+         if (buf.readableBytes() >= 4) { // ← unnecessary AND semantically wrong
            buf.readUnsignedInt(); // device time
+         }
+         if (buf.readableBytes() >= 2) { // ← correct guard (the actual fix)
            decodeStatus(position, buf.readUnsignedShort());
+         }
+       }
    }
}

```

Fig. 37. LLM-generated patch for *RegressionBug-101*, illustrating the over-repair failure mode where six guards are added across four read operations instead of the single required guard.

²⁵ Available at: <https://brojackvn.github.io/RegMiner4APR-Homepage/#/bug/RegressionBug-122/RegressionBug-122>

²⁶ Available at: <https://brojackvn.github.io/RegMiner4APR-Homepage/#/bug/RegressionBug-101/RegressionBug-101>

SUMMARY

Our analysis identifies four categories of LLM repair failures: Wrong Diagnosis (44.07%), Weak Test Suite (23.73%), Unreachable Fix (20.34%), and Over-repair (11.86%). Wrong Diagnosis is the most prevalent failure mode, arising from limited reasoning capability or misdirection from error signals. These findings suggest that future repair approaches should focus on improving root cause reasoning, strengthening test suite quality, and incorporating broader contextual knowledge beyond the method body.

RQ3 Impact of bug-inducing change information on APR

Our experimental results demonstrate that incorporating bug-inducing change (BIC) information significantly enhances the effectiveness of prompt-based APR techniques in repairing regression errors. On the Java benchmark, ChatGPT-4o under the conversational configuration with BIC context achieves the highest number of correct repairs, successfully fixing 29 out of 150 bugs (19.33%). On the Python benchmark, the same configuration achieves 10 out of 50 correct repairs (20.00%), consistent with the trends observed in Java and supporting the generalizability of our findings across programming languages. In total, across both benchmarks, this configuration correctly repairs 39 out of 200 bugs (19.50%), representing a 1.6× improvement over the best-performing configuration without BIC.

7 RQ4: Ablation Study

Building on the findings of RQ3, we conduct an ablation study to isolate and evaluate the individual contribution of each contextual element within the bug-inducing change (BIC) information. We dissect the best-performing configuration from RQ3, specifically conversational ChatGPT-4o, as the base model. We evaluate the following four configurations: (i) *w/o BIC*, where no additional context is provided beyond the buggy program and failing tests; (ii) *Commit message only*, where only the commit message of the bug-inducing commit is appended to the prompt; (iii) *Code changes only*, where only the code changes to the buggy function and the list of changed file names are included, without the commit message; and (iv) *Full BIC*, where the complete bug-inducing change information is provided, combining both the commit message and the code-level changes described in (ii) and (iii).

Table 7. Ablation Study Results on Java regression bugs of the REGRESSIONBUG4APR benchmark

| Configurations
(Conversational ChatGPT-4o as a base strategy) | #Plausible Patches
(Plausible Rate %) | #Correct Patches
(Correct Rate %) | Precision (%) |
|--|--|--------------------------------------|---------------|
| w/o BIC | 60
(40.00%) | 18
(12.00%) | 30.00% |
| + Commit message | 67
(44.67%) | 21
(14.00%) | 31.34% |
| + Code changes | 70
(46.67%) | 25
(16.67%) | 35.71% |
| Full BIC | 72
(48.00%) | 29
(19.33%) | 40.28% |

Table 7 presents the ablation study results on the Java benchmark. Our results reveal several key insights. First, each contextual component contributes positively and incrementally to repair performance, confirming that the improvements observed in RQ3 are not solely attributable to a single element. Second, code-level change information consistently provides a larger benefit than the commit message alone: adding code changes yields 25 correct patches

(35.71% precision) compared to 21 (31.34% precision) for commit message only, both representing meaningful gains over the w/o BIC baseline of 18 correct patches (30.00% precision). This suggests that concrete code context plays a more decisive role in guiding the model toward correct patches than natural-language commit descriptions. Third, the two components are complementary: combining both yields the best outcome of 29 correct patches (40.28% precision), an improvement of 11 correct patches over the w/o BIC baseline, demonstrating that commit messages and code changes capture different, non-redundant signals. The commit message conveys the high-level intent of the change, while the code diff provides structural and syntactic grounding that narrows the hypothesis space for the model.

Table 8. Ablation Study Results on Python regression bugs of the REGRESSIONBUG4APR benchmark.

| Configurations
(Conversational ChatGPT-4o as a base strategy) | #Plausible Patches
(Plausible Rate %) | #Correct Patches
(Correct Rate %) | Precision (%) |
|--|--|--------------------------------------|---------------|
| w/o BIC | 29
(58.00%) | 6
(12.00%) | 20.69% |
| + Commit message | 33
(66.00%) | 7
(14.00%) | 21.21% |
| + Code changes | 32
(64.00%) | 9
(18.00%) | 28.12% |
| Full BIC | 35
(70.00%) | 10
(20.00%) | 28.57% |

Table 8 presents the ablation study results on the Python benchmark. The results are largely consistent with the trends observed in the Java benchmark, providing further evidence for the generalizability of our findings. Adding code changes yields 9 correct patches (28.12% precision) compared to 7 (21.21% precision) for commit message only, both representing meaningful gains over the w/o BIC baseline of 6 correct patches (20.69% precision). The full BIC combination yields the best outcome of 10 correct patches (28.57% precision), an improvement of 4 correct patches over the w/o BIC baseline, suggesting that the two contextual components are complementary across programming languages.

RQ4 Ablation Study

Our ablation study reveals that each contextual component within the bug-inducing change information contributes positively and incrementally to repair performance. Code-level changes consistently provide a larger gain than commit messages alone across both benchmarks. On the REGRESSIONBUG4APR benchmark (200 bug instances across Java and Python), the full BIC combination yields the best overall outcome of 39 correct patches (36.45% precision), compared to 34 correct patches (33.33% precision) for code changes only, 28 correct patches (28.00% precision) for commit message only, and 24 correct patches (26.97% precision) for the w/o BIC baseline. These results suggest that commit messages and code changes capture complementary, non-redundant signals that together enhance repair effectiveness.

8 Discussion

8.1 Lesson Learned

In this subsection, we highlight key lessons learned through our experiments and analysis that can drive future research in the field.

Enhanced APR Workflow with Bug-inducing change Information: Our empirical results highlight the significant impact of incorporating regression-inducing context on repair effectiveness. Conventionally, APR workflows operate by taking a buggy program and test suite, and attempting to generate a minimal change that pass all test cases. However, our findings suggest that enriching this workflow with additional context, specifically by detecting whether a bug is regression-related and extracting the corresponding bug-inducing commit, can substantially enhance repair outcomes. This extension is particularly beneficial in the context of continuous integration (CI) systems, where regression bugs emerge frequently and need to be addressed efficiently at scale. To further illustrate this applicability, we present concrete CI use cases in Section 8.2, demonstrating how our approach can be integrated into real-world CI pipelines and showing its effectiveness on two case studies of regression bugs detected through CI systems.

Context-Aware Use of Reversion Operators: Our qualitative analysis reveals several cases where *fully or partially reverting to previously correct statements* operators result in plausible but incorrect patches. These findings suggest that while reversion-based repair operators can be effective patterns, their successful application requires a proper understanding of the bug context. Blindly applying such operators, without considering the underlying cause of the regression, can lead to overfitting and semantically incorrect fixes. This also suggest that template-based APR techniques must understand the context to determine whether reversion patterns are appropriate.

8.2 Regression Repair Use Case In CI Settings

In a typical continuous integration (CI) pipeline, e.g., Jenkins ²⁷ or Github Actions ²⁸, every code commit triggers an automated build and regression test suite execution. When a regression is detected, i.e., a previously passing test begins to fail after a recent commit, developers must manually investigate the root cause and implement a fix, which is both time-consuming and disrupts the development workflow. In this setting, our context-aware APR workflow can be naturally integrated as an additional stage in the CI pipeline. Once the bug-inducing commit is identified, e.g., via automated tools such as SZZ, git bisect or RegMiner, the corresponding code changes, commit messages and failing regression tests can be extracted and provided to our repair tool. Our tool then generates candidate patches that explicitly account for the regression-inducing modification. These candidate patches can be automatically validated within the same CI pipeline by re-running the test suite. If a patch successfully restores the failing test while preserving previously passing behavior, it can be presented to developers as a repair suggestion (e.g., as part of CI feedback or a pull request comment). This enables a more automated and efficient regression repair process, reducing manual debugging effort and providing timely feedback during continuous development.

To further validate the practical applicability of our approach, we study two real-world regression bugs detected through CI pipelines:

Case 1: Vectorz ²⁹ (bug-fixing commit `cc07d6f6cc`, bug-inducing commit `3eee45ec7a`): Vectorz is an open-source Java library for high-performance vector and matrix arithmetic. The bug-inducing commit ("*extra index checks on selectClone*") introduced a regression in the `selectClone` method of `AVector.java` by incorrectly writing `checkIndex(i)`, which validates the loop counter, instead of `checkIndex(ix)`, which should validate the caller-provided index. This caused the `testSelect` regression test to fail with an `IndexOutOfBoundsException` for valid inputs. Both conversational APR with ChatGPT-4o with and without BIC information successfully generated the correct patch, i.e., replacing `checkIndex(i)` with `checkIndex(ix)`, restoring the failing test while preserving all other existing tests. This suggests

²⁷ Available at: <https://www.jenkins.io>

²⁸ Available at: <https://github.com/features/actions>

²⁹ Available at: <https://github.com/mikera/vectorz>

that for straightforward regression bugs where the fault is clearly localized, the test failure information alone may be sufficient to guide the repair.

Case 2: Traccar³⁰ (bug-fixing commit *30d5a99aab*, bug-inducing commit *a6b8f7f7ef*). Traccar is an open-source Java-based GPS tracking platform supporting over 200 GPS protocols and more than 2,000 device models. The bug-inducing commit (“*Decode course for Aquila protocol*”) extended `AquilaProtocolDecoder.java` by introducing bitwise logic to derive a composite course index from four binary bits, but guarded the bearing assignment with `if (course > 0)`, omitting an upper-bound constraint, thereby permitting out-of-range indices (values 9–15) to produce invalid bearings outside the valid compass range of 0°–359°. This caused the `testDecode` regression test to fail for protocol messages whose course bits encode an invalid value. Without BIC information, conversational APR with ChatGPT-4o failed to identify the valid range constraint and did not produce a plausible patch. Incorporating BIC information enabled our tool to correctly identify the root cause, that the feature was designed around exactly eight valid directional octants (indices 1–8), and generate the correct patch by augmenting the guard condition from `if (course > 0) {position.setCourse((course - 1) * 45);}` to `if (course >= 0 && course <= 7) {position.setCourse(course * 45);}`, restoring the failing test while preserving all previously passing behavior. Notably, the generated patch is syntactically equivalent to the human-written fix.

These two case studies demonstrate complementary aspects of our approach: (i) it is directly applicable to real-world CI settings where regression bugs are naturally detected through automated pipelines; and (ii) bug-inducing change information is crucial for effective repair in non-trivial cases, as illustrated by the contrasting results between the two configurations in Case 2.

8.3 Limitations and Future Directions

While our empirical results demonstrate the promising potential of LLMs in repairing regression bugs, they also reveal several limitations across both fine-tuning-based and prompt-based approaches, as well as challenges that persist even when BIC context is provided. We discuss these limitations below and outline concrete directions for future work.

Fine-tuning-based approaches exhibit several shortcomings. First, they do not incorporate test failure information during inference, relying solely on learned bug-fix patterns, which limits their ability to reason about the specific failure context. Second, they cannot readily incorporate auxiliary information such as BIC context without retraining, limiting their adaptability to regression-specific repair scenarios. Third, as illustrated by *RegressionBug-76*, these models either produce structurally malformed patches or syntactically correct but semantically meaningless ones, indicating that learned patterns are insufficient when the root cause requires library-specific knowledge. Fourth, CodeGen and InCoder achieve significantly lower plausible rates (7.33%–9.33%) compared to prompt-based approaches, suggesting that their generated patches frequently fail basic compilation or test execution. Fifth, smaller models (CodeGen-2B vs. CodeGen-6B, InCoder-1B vs. InCoder-6B) do not show consistent improvement with scale, suggesting the bottleneck lies in the training data rather than model capacity.

Prompt-based approaches also exhibit some limitations. First, conversational strategies tend to overfit to failing test cases through iterative feedback, producing a higher number of plausible patches but relatively few correct ones. Second, as illustrated by *RegressionBug-76*, prompt-based approaches conflate the regression fault with unrelated patterns from general domain knowledge, leading to semantically plausible but factually incorrect patches. Third,

³⁰ Available at: <https://github.com/traccar/traccar>

without BIC context, prompt-based techniques cannot identify the root cause of the regression when the test failure does not directly reflect the underlying fault.

Remaining challenges with BIC context. Even the best-performing configuration correctly repairs only 39 out of 200 bugs, indicating that there remains substantial room for improvement. Specifically, Remote bugs remain the most challenging category even when BIC information is provided, with the fix rate improving only modestly from 9.4% to 16.0%. Furthermore, reversion strategies must be applied judiciously, as blindly reverting to previous code can produce plausible but semantically incorrect patches when the bug-inducing commit contains a mixture of intended and unintended changes.

Based on the limitations above, we identify the following concrete directions for future LLM-based regression repair research: (i) incorporating test failure feedback and BIC context during fine-tuning training, enabling regression-aware repair beyond static bug-fix patterns; (ii) integrating domain-specific and library-level knowledge, either through retrieval-augmented generation or knowledge-enhanced prompting, to address cases where the correct fix cannot be inferred from the available context alone; (iii) developing patch diversity enforcement mechanisms to reduce overfitting to the test suite in conversational repair; and (iv) designing repair approaches that can reason across multiple program versions and distinguish intended from unintended modifications within a bug-inducing commit, to better handle Remote regression bugs and complex reversion scenarios.

8.4 Experimental Cost Report

Running APR experiments using LLMs requires substantial computational resources. In our study, experiments involving RepairLLaMA, InCoder-1B/6B, and CodeGen-2B/6B were conducted using an NVIDIA A100 GPU with 80GB of VRAM. To prevent out-of-memory (OOM) crashes observed during generation, we limited the beam size to 10.

In contrast, experiments using ChatGPT models (ChatGPT-4o and ChatGPT-Turbo-3.5) did not require local computational resources, as these models were accessed via OpenAI’s API. However, using these APIs incurs direct monetary costs, as users are charged based on the number of input and output tokens processed by the model. While the exact cost depends on usage, it is worth noting that these models still rely on significant backend computing infrastructure, even if this burden is offloaded from the researcher to the service provider. As of the time of our implementation, the cost for ChatGPT-4o was \$0.0025 per 1,000 input tokens and \$0.01 per 1,000 output tokens. For ChatGPT-Turbo-3.5, the cost was \$0.0005 per 1,000 input tokens and \$0.0015 per 1,000 output tokens.

Figure 38 presents the token cost of patch generation per bug across different configurations on the REGRESSION-BUG4APR benchmark. For zero-shot prompting, the model generates up to 10 patches per bug and stops once a plausible patch is found. In the conversational strategy, the model performs up to 10 attempts per bug, with each attempt involving up to 5 rounds of interaction guided by test case feedback, terminating upon generation of a plausible patch.

Overall, the conversational strategy incurs a higher cost than zero-shot prompting due to the multiple rounds of interaction involved. It also exhibits higher cost variance, with several outliers indicating that certain bugs require significantly more rounds of interaction to repair. Regarding the effect of BIC context, an interesting divergence is observed between the two models. For ChatGPT-Turbo-3.5, incorporating BIC context leads to slightly higher costs, as longer prompts increase the number of input tokens. In contrast, for ChatGPT-4o, incorporating BIC context reduces both the median cost and cost variance, as BIC helps the model generate plausible patches in fewer attempts, making it not only more effective but also more cost-efficient. This divergence suggests that the cost-effectiveness of BIC context depends on the underlying model capability, with stronger models being better able to leverage the additional context to reduce repair effort.

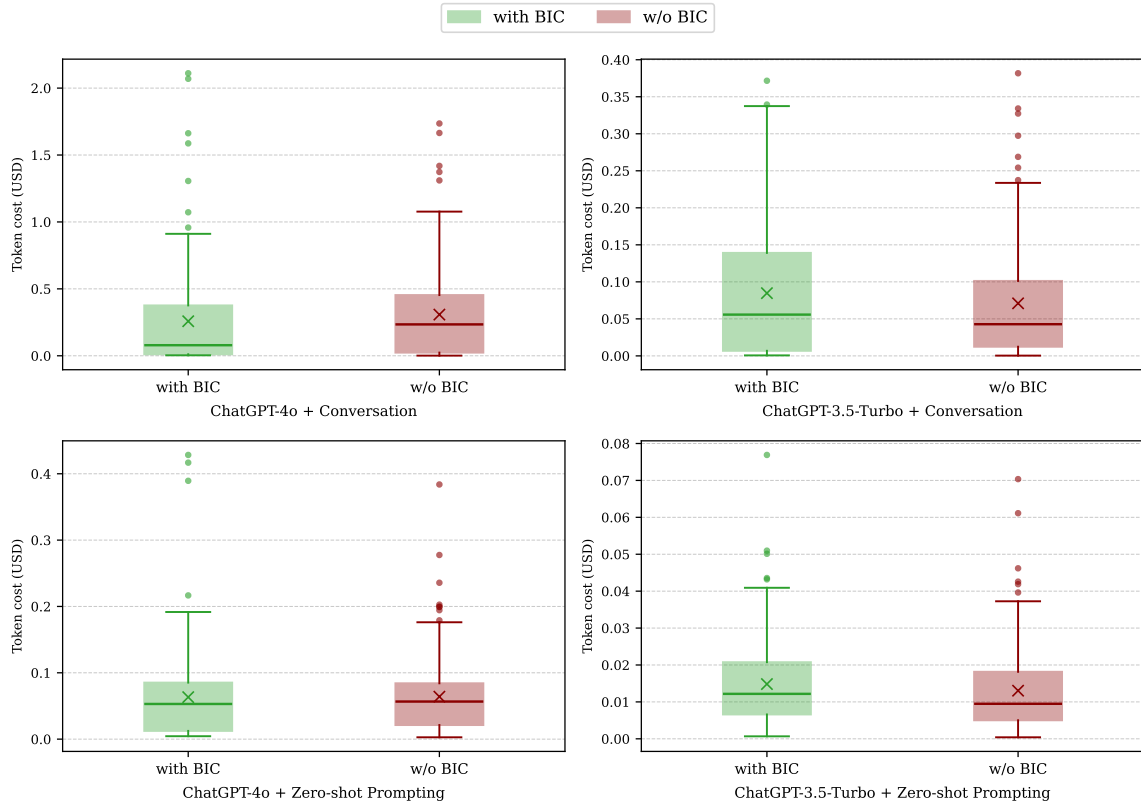


Fig. 38. Token cost of patch generation per bug across different methods.

8.5 Threat to Validity

We anticipate that there are the following threats to the validity of our findings.

8.5.1 Internal Validity. An internal threat comes from the manual validation used to determine the correctness of the plausible patches. To mitigate this threat, we conduct a thorough examination of each patch following prior works [76], along with careful double-checking of the results. Still, the assessment of patch correctness remains an active area of research, and further investigation is needed to refine assessment methodologies. Another internal threat concerns the manual bug categorization, which involves analyzing, dissecting, and classifying developer-written patches according to established repair operators defined in prior works [54, 63]. To mitigate this risk, we involved an author with over three years of experience in Java and Python programming to perform this task, which was then double-checked by another author. Since these repair operators represent fundamental constructs that are common across imperative programming languages, the same set of operators is applied to both Java and Python patches, and the categorization process can be considered reasonably reliable across both languages.

8.5.2 External Validity. One external threat relates to the potential exposure of our dataset to an LLM’s training corpus. However, prior work [24] has shown that data leakage is less of a concern for APR compared to other code-related tasks. This is because LLM training corpora typically contain, at most, individual versions of a program, either buggy or

fixed, but not aligned bug-fix pairs. Without such aligned data, LLMs are unlikely to learn APR tasks directly from training. Another external threat concerns the generalizability of our findings. In this study, we evaluated several APR techniques using 200 regression bugs collected from widely-used Java and Python GitHub repositories. While the dataset is not large, the findings could be further strengthened by expanding to a larger and more diverse set of programs with additional real-world bugs. Nevertheless, we mitigate this threat through the analysis presented in RQ1, which demonstrates substantial diversity in both fix scope and repair operators. Furthermore, our evaluation across both Java and Python regression bugs provides preliminary evidence for the generalizability of our findings across programming languages. Moreover, the benchmark has been designed with extensibility, allowing future expansion as more regression bugs are identified and validated.

8.5.3 Construct Validity. One construct validity threat concerns the experimental setup used in our empirical study. To ensure a fair comparison, we adopt the original configurations (e.g., architectures and parameters) provided by the authors of the respective techniques, preserving their internal structures. The only modification made was to the sampling size, which was adjusted to control computational cost while keeping the core techniques intact. For prompt-based techniques, we use the same prompt templates and configurations across both Java and Python regression bugs, ensuring that any observed differences in repair effectiveness are attributable to the nature of the bugs rather than differences in experimental setup.

9 Conclusion

In this study, we conduct an empirical study of automated program repair (APR) techniques on regression errors, including both traditional and LLM-based approaches. The evaluation is performed using our high-quality benchmark, REGRESSIONBUG4APR, which comprises 200 real-world regression bugs collected from widely-used Java and Python GitHub repositories. Experimental results reveal that traditional APR tools fail to repair any of the bugs, whereas LLM-based techniques demonstrate promising performance.

Motivated by the effectiveness of LLM-based approaches, we focus on prompt-based techniques due to their flexibility and efficiency compared to fine-tuning-based methods. We specifically investigate whether incorporating regression-specific context, particularly bug-inducing change (BIC) information, can enhance repair effectiveness. Our results show substantial improvements in repair performance when BIC context is included. Specifically, the best-performing configuration, conversational ChatGPT-4o with full BIC information, successfully repairs 39 out of 200 regression bugs, representing a $1.6\times$ improvement over the best-performing configuration without BIC context. Our qualitative analysis reveals that BIC information provides crucial context that helps models better identify the root cause of regressions and decide whether to fully or partially revert to previously correct statements. It also narrows the fix scope, improving fault localization and reducing unnecessary code modifications. In addition, our breakdown by regression bug type shows that Remote bugs are consistently the most challenging category to repair across both languages and all evaluated configurations, while Local bugs, being the most prevalent category, account for the majority of correct repairs.

Furthermore, our ablation study reveals that each contextual component of BIC information contributes positively and incrementally to repair performance, with code-level changes yielding a larger gain than commit messages alone, and that the two components capture complementary, non-redundant signals that together enhance repair effectiveness. These findings are consistent across both Java and Python benchmarks, supporting the generalizability of our conclusions.

Reflecting on the findings, we draw two key insights for future work: improving regression bug repair and leveraging additional context in APR. First, our study suggests that exposing models to how past code changes introduced a regression provides valuable cues for understanding root causes and generating correct fixes. This highlights a new opportunity to guide general-purpose LLMs in identifying regression patterns and applying suitable repairs. However, our analysis also reveals that even with BIC context, certain regression bugs, such as Remote bugs and those requiring library-specific knowledge, remain difficult to repair, suggesting that BIC information alone is not sufficient and must be complemented with domain-specific or cross-component reasoning capabilities. Second, building on recent advances in LLMs, future APR research may benefit from systematically mining and integrating additional context, such as software evolution, into model training. In this direction, future fine-tuning-based approaches should incorporate test failure feedback and BIC context directly into the training process, rather than relying solely on static bug-fix patterns, enabling models to perform regression-aware repair while retaining the efficiency benefits of fine-tuning.

References

- [1] 2025. RegRepair: Prompt-based Program Repair for Regression Bugs. <https://github.com/brojackvn/RegressionBug4APR-Repair>.
- [2] 2025. RegressionBug4APR Benchmark (Java portion). <https://github.com/brojackvn/RegressionBug4APR-Benchmark-Java>.
- [3] 2025. RegressionBug4APR Framework (Java portion). <https://github.com/brojackvn/RegressionBug4APR-Framework-Java>.
- [4] 2025. RegressionBug4APR Framework (Python portion). <https://github.com/brojackvn/RegressionBug4APR-Framework-Python>.
- [5] 2025. RegressionBug4APR Homepage. <https://brojackvn.github.io/RegressionBug4APR-Homepage>.
- [6] 2025. RegressionBug4APR Mining Tool (Python portion). <https://github.com/brojackvn/RegressionBug4APR-Mining-Python>.
- [7] 2025. RegressionBug4APR Regression Validation Tool. <https://github.com/brojackvn/RegressionBug4APR-Framework-Java/tree/main/regression-validation>.
- [8] Abdulaziz Alhefdhi, Hoa Khanh Dam, Thanh Le-Cong, Bach Le, and Aditya Ghose. 2025. Adversarial patch generation for automated program repair. *Software Quality Journal* 33, 1 (Jan. 2025), 23 pages. doi:10.1007/s11219-025-09709-4
- [9] Gabin An, Jingun Hong, Naryeong Kim, and Shin Yoo. 2023. Fonte: Finding bug inducing commits from failures. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 589–601.
- [10] Earl T Barr, Yuriy Brun, Premkumar Devanbu, Mark Harman, and Federica Sarro. 2014. The plastic surgery hypothesis. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*. 306–317.
- [11] Moritz Beller, Georgios Gousios, and Andy Zaidman. 2017. Travistorrent: Synthesizing travis ci and github for full-stack research on continuous integration. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. IEEE, 447–450.
- [12] Marcel Böhme, Bruno C d S Oliveira, and Abhik Roychoudhury. 2013. Regression tests to expose change interaction errors. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering*. 334–344.
- [13] Marcel Böhme and Abhik Roychoudhury. 2014. Corebench: Studying complexity of regression errors. In *Proceedings of the 2014 international symposium on software testing and analysis*. 105–115.
- [14] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2025. Repairagent: An autonomous, llm-based agent for program repair. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2188–2200.
- [15] Jacob Cohen. 1960. A coefficient of agreement for nominal scales. *Educational and psychological measurement* 20, 1 (1960), 37–46.
- [16] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. *Advances in neural information processing systems* 36 (2023), 10088–10115.
- [17] Thomas Durieux and Rui Abreu. 2019. Critical review of bugswarm for fault localization and program repair. *arXiv preprint arXiv:1905.09375* (2019).
- [18] Hadeel Eladawy, Claire Le Goues, and Yuriy Brun. 2024. Automated program repair, what is it good for? not absolutely nothing!. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
- [19] Emelie Engström and Per Runeson. 2010. A qualitative survey of regression testing practices. In *Product-Focused Software Process Improvement: 11th International Conference, PROFES 2010, Limerick, Ireland, June 21-23, 2010. Proceedings 11*. Springer, 3–16.
- [20] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. 2022. InCoder: A generative model for code infilling and synthesis. *arXiv preprint arXiv:2204.05999* (2022).
- [21] Xiang Gao, Yannic Noller, and Abhik Roychoudhury. 2022. Program repair. *arXiv preprint arXiv:2211.12787* (2022).
- [22] Péter Gyimesi, Béla Vancsics, Andrea Stocco, Davood Mazinanian, Árpád Beszédes, Rudolf Ferenc, and Ali Mesbah. 2019. Bugsjs: a benchmark of javascript bugs. In *2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST)*. IEEE, 90–101.
- [23] Kai Huang, Jian Zhang, Xinlei Bao, Xu Wang, and Yang Liu. 2025. Comprehensive Fine-Tuning Large Language Models of Code for Automated Program Repair. *IEEE Transactions on Software Engineering* (2025).

- [24] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of code language models on automated program repair. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1430–1442.
- [25] Nan Jiang, Thibaud Lutellier, Yiling Lou, Lin Tan, Dan Goldwasser, and Xiangyu Zhang. 2023. Knod: Domain knowledge distilled tree decoder for automated program repair. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1251–1263.
- [26] Nan Jiang, Thibaud Lutellier, and Lin Tan. 2021. Cure: Code-aware neural machine translation for automatic program repair. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1161–1173.
- [27] René Just, Darioush Jalali, and Michael D Ernst. 2014. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 international symposium on software testing and analysis*. 437–440.
- [28] Vinay Kabadi, Dezhen Kong, Siyu Xie, Lingfeng Bao, Gede Artha Azriadi Prana, Tien-Duy B Le, Xuan-Bach D Le, and David Lo. 2023. The Future Can't Help Fix The Past: Assessing Program Repair In The Wild. In *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 50–61.
- [29] Manisha Khattar, Yash Lamba, and Ashish Sureka. 2015. Sarathi: Characterization study on regression bugs and identification of regression bug inducing changes: A case-study on google chromium project. In *Proceedings of the 8th india software engineering conference*. 50–59.
- [30] Xuan-Bach D Le, Duc-Hiep Chu, David Lo, Claire Le Goues, and Willem Visser. 2017. S3: syntax-and semantic-guided repair synthesis via programming by examples. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*. 593–604.
- [31] Xuan Bach D. Le, David Lo, and Claire Le Goues. 2016. History Driven Program Repair. In *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, Vol. 1. 213–224. doi:10.1109/SANER.2016.76
- [32] Xuan-Bach D Le, Ferdian Thung, David Lo, and Claire Le Goues. 2018. Overfitting in semantics-based automated program repair. In *Proceedings of the 40th international conference on software engineering*. 163–163.
- [33] Thanh Le-Cong, Bach Le, and Toby Murray. 2024. Semantic-guided Search for Efficient Program Repair with Large Language Models. *arXiv preprint arXiv:2410.16655* (2024).
- [34] Thanh Le-Cong, Dat Nguyen, Bach Le, and Toby Murray. 2025. Towards reliable evaluation of neural program repair with natural robustness testing. *ACM Transactions on Software Engineering and Methodology* 34, 7 (2025), 1–44.
- [35] Claire Le Goues, Neal Holtschulte, Edward K Smith, Yuriy Brun, Premkumar Devanbu, Stephanie Forrest, and Westley Weimer. 2015. The ManyBugs and IntroClass benchmarks for automated repair of C programs. *IEEE Transactions on Software Engineering* 41, 12 (2015), 1236–1256.
- [36] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2011. Genprog: A generic method for automatic software repair. *Ieee transactions on software engineering* 38, 1 (2011), 54–72.
- [37] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. 2019. Automated program repair. *Commun. ACM* 62, 12 (2019), 56–65.
- [38] Fengjie Li, Jiajun Jiang, Jiajun Sun, and Hongyu Zhang. 2024. Hybrid automated program repair by combining large language models and program analysis. *ACM Transactions on Software Engineering and Methodology* (2024).
- [39] Kui Liu, Anil Koyuncu, Dongsun Kim, and Tegawendé F Bissyandé. 2019. TBar: Revisiting template-based automated program repair. In *Proceedings of the 28th ACM SIGSOFT international symposium on software testing and analysis*. 31–42.
- [40] Kui Liu, Li Li, Anil Koyuncu, Dongsun Kim, Zhe Liu, Jacques Klein, and Tegawendé F Bissyandé. 2021. A critical review on the evaluation of automated program repair systems. *Journal of Systems and Software* 171 (2021), 110817.
- [41] Yue Liu, Thanh Le-Cong, Ratnadira Widyasari, Chakkrit Tantithamthavorn, Li Li, Xuan-Bach D Le, and David Lo. 2024. Refining chatgpt-generated code: Characterizing and mitigating code quality issues. *ACM Transactions on Software Engineering and Methodology* 33, 5 (2024), 1–26.
- [42] David Lo, Nachiappan Nagappan, and Thomas Zimmermann. 2015. How practitioners perceive the relevance of software engineering research. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*. 415–425.
- [43] Wenqiang Luo, Jacky Keung, Boyang Yang, He Ye, Claire Le Goues, Tegawendé F. Bissyandé, Haoye Tian, and Xuan Bach D. Le. 2025. When Fine-Tuning LLMs Meets Data Privacy: An Empirical Study of Federated Learning in LLM-Based Program Repair. *ACM Trans. Softw. Eng. Methodol.* (May 2025). doi:10.1145/3733599 Just Accepted.
- [44] Fernanda Madeiral, Simon Urli, Marcelo Maia, and Martin Monperrus. 2019. Bears: An extensible java bug benchmark for automatic program repair studies. In *2019 IEEE 26th international conference on software analysis, evolution and reengineering (SANER)*. IEEE, 468–478.
- [45] Matias Martinez and Martin Monperrus. 2016. Astor: A program repair library for java. In *Proceedings of the 25th international symposium on software testing and analysis*. 441–444.
- [46] Matias Martinez and Martin Monperrus. 2018. Ultra-large repair search space with automatically mined templates: The cardumen mode of astor. In *Search-Based Software Engineering: 10th International Symposium, SSBSE 2018, Montpellier, France, September 8-9, 2018, Proceedings 10*. Springer, 65–86.
- [47] Sergey Mechtaev, Jooyong Yi, and Abhik Roychoudhury. 2016. Angelix: Scalable multiline program patch synthesis via symbolic analysis. In *Proceedings of the 38th international conference on software engineering*. 691–701.
- [48] Martin Monperrus. 2018. *The living review on automated program repair*. Ph. D. Dissertation. HAL Archives Ouvertes.
- [49] Hoang Duong Thien Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. 2013. Semfix: Program repair via semantic analysis. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 772–781.
- [50] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474* (2022).

- [51] Dor Nir, Shmuel Tyszberowicz, and Amiram Yehudai. 2008. Locating regression bugs. In *Hardware and Software: Verification and Testing: Third International Haifa Verification Conference, HVC 2007, Haifa, Israel, October 23-25, 2007. Proceedings 3*. Springer, 218–234.
- [52] Yannic Noller, Ridwan Shariffdeen, Xiang Gao, and Abhik Roychoudhury. 2022. Trust enhancement issues in program repair. In *Proceedings of the 44th International Conference on Software Engineering*. 2228–2240.
- [53] Akira K Onoma, Wei-Tek Tsai, Mustafa Poonawala, and Hiroshi Suganuma. 1998. Regression testing in an industrial environment. *Commun. ACM* 41, 5 (1998), 81–86.
- [54] Kai Pan, Sunghun Kim, and E James Whitehead. 2009. Toward an understanding of bug fix patterns. *Empirical Software Engineering* 14 (2009), 286–315.
- [55] Zichao Qi, Fan Long, Sara Achour, and Martin Rinard. 2015. An analysis of patch plausibility and correctness for generate-and-validate patch generation systems. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis*. 24–36.
- [56] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).
- [57] Haifeng Ruan, Yuntong Zhang, and Abhik Roychoudhury. 2024. Specrover: Code intent extraction via llms. *arXiv preprint arXiv:2408.02232* (2024).
- [58] Ripon K Saha, Yingjun Lyu, Wing Lam, Hiroaki Yoshida, and Mukul R Prasad. 2018. Bugs. jar: A large-scale, diverse dataset of real-world java bugs. In *Proceedings of the 15th international conference on mining software repositories*. 10–13.
- [59] Ridwan Shariffdeen, Martin Mirchev, Yannic Noller, and Abhik Roychoudhury. 2023. Cerberus: a Program Repair Framework. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 73–77.
- [60] André Silva, Sen Fang, and Martin Monperrus. 2023. Repairllama: Efficient representations and fine-tuned adapters for program repair. *arXiv preprint arXiv:2312.15698* (2023).
- [61] Jacek Sliwinski, Thomas Zimmermann, and Andreas Zeller. 2005. When do changes induce fixes? *ACM sigsoft software engineering notes* 30, 4 (2005), 1–5.
- [62] Edward K Smith, Earl T Barr, Claire Le Goues, and Yuriy Brun. 2015. Is the cure worse than the disease? overfitting in automated program repair. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*. 532–543.
- [63] Victor Sobreira, Thomas Durieux, Fernanda Madeiral, Martin Monperrus, and Marcelo de Almeida Maia. 2018. Dissection of a bug dataset: Anatomy of 395 patches from defects4j. In *2018 IEEE 25th international conference on software analysis, evolution and reengineering (SANER)*. IEEE, 130–140.
- [64] Xuezhi Song, Yun Lin, Siang Hwee Ng, Yijian Wu, Xin Peng, Jin Song Dong, and Hong Mei. 2022. Regminer: towards constructing a large regression dataset from code evolution history. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. 314–326.
- [65] Donna Spencer. 2009. *Card sorting: Designing usable categories*. Rosenfeld Media.
- [66] Shin Hwei Tan and Abhik Roychoudhury. 2015. relifix: Automated repair of software regressions. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 1. IEEE, 471–482.
- [67] Shin Hwei Tan, Jooyong Yi, Sergey Mechtaev, Abhik Roychoudhury, et al. 2017. Codeflaws: a programming competition benchmark for evaluating automated program repair tools. In *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*. IEEE, 180–182.
- [68] David A Tomassi, Naji Dmeiri, Yichen Wang, Antara Bhowmick, Yen-Chuan Liu, Premkumar T Devanbu, Bogdan Vasilescu, and Cindy Rubio-González. 2019. Bugswarm: Mining and continuously growing a dataset of reproducible failures and fixes. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 339–349.
- [69] Ming Wen, Rongxin Wu, Yepang Liu, Yongqiang Tian, Xuan Xie, Shing-Chi Cheung, and Zhendong Su. 2019. Exploring and exploiting the correlations between bug-inducing and bug-fixing commits. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 326–337.
- [70] Ratnadira Widayarsi, Sheng Qin Sim, Camellia Lok, Haodi Qi, Jack Phan, Qijin Tay, Constance Tan, Fiona Wee, Jodie Ethelda Tan, Yuheng Yieh, et al. 2020. Bugsinpy: a database of existing bugs in python programs to enable controlled testing and debugging studies. In *Proceedings of the 28th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*. 1556–1560.
- [71] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1482–1494.
- [72] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated program repair via conversation: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 819–831.
- [73] Jiahong Xiang, Xiaoyang Xu, Fanchu Kong, Mingyuan Wu, Zizheng Zhang, Haotian Zhang, and Yuqun Zhang. 2024. How far can we go with practical function-level program repair? *arXiv preprint arXiv:2404.12833* (2024).
- [74] Guanping Xiao, Zheng Zheng, Bo Jiang, and Yulei Sui. 2019. An empirical study of regression bug chains in linux. *IEEE Transactions on Reliability* 69, 2 (2019), 558–570.
- [75] Guanping Xiao, Zheng Zheng, Beibei Yin, Kishor S Trivedi, Xiaoting Du, and Kai-Yuan Cai. 2019. An empirical study of fault triggers in the Linux operating system: An evolutionary perspective. *IEEE Transactions on Reliability* 68, 4 (2019), 1356–1383.
- [76] Junjielong Xu, Ying Fu, Shin Hwei Tan, and Pinjia He. 2024. Aligning the Objective of LLM-based Program Repair. *arXiv preprint arXiv:2404.08877* (2024).
- [77] Jifeng Xuan, Matias Martinez, Favio Demarco, Maxime Clement, Sebastian Lamelas Marcote, Thomas Durieux, Daniel Le Berre, and Martin Monperrus. 2016. Nopol: Automatic repair of conditional statement bugs in java programs. *IEEE Transactions on Software Engineering* 43, 1 (2016), 34–55.

- [78] Boyang Yang, Haoye Tian, Jiadong Ren, Shunfu Jin, Yang Liu, Feng Liu, and Bach Le. 2025. Enhancing Repository-Level Software Repair via Repository-Aware Knowledge Graphs. arXiv:2503.21710 [cs.SE] <https://arxiv.org/abs/2503.21710>
- [79] He Ye, Matias Martinez, and Martin Monperrus. 2022. Neural program repair with execution-based backpropagation. In *Proceedings of the 44th international conference on software engineering*. 1506–1518.
- [80] Xin Yin, Chao Ni, Shaohua Wang, Zhenhao Li, Limin Zeng, and Xiaohu Yang. 2024. Thinkrepair: Self-directed automated program repair. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1274–1286.
- [81] Zuoning Yin, Ding Yuan, Yuanyuan Zhou, Shankar Pasupathy, and Lakshmi Bairavasundaram. 2011. How do fixes become bugs?. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*. 26–36.
- [82] Yuan Yuan and Wolfgang Banzhaf. 2018. Arja: Automated repair of java programs via multi-objective genetic programming. *IEEE Transactions on software engineering* 46, 10 (2018), 1040–1067.
- [83] Andreas Zeller. 1999. Yesterday, my program worked. Today, it does not. Why? *ACM SIGSOFT Software engineering notes* 24, 6 (1999), 253–267.
- [84] Qihao Zhu, Zeyu Sun, Yuan-an Xiao, Wenjie Zhang, Kang Yuan, Yingfei Xiong, and Lu Zhang. 2021. A syntax-guided edit decoder for neural program repair. In *Proceedings of the 29th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*. 341–353.